

BIG-IP Layer 7 DoS Lab Guide

Modules 1-3 - iRules, LTM Policies, ASM/Advanced WAF

F5 Lab

Oct 06, 2026

1	Lab Environment	3
2	Prerequisites	5
3	Accessing the lab environment	7
3.1	Lab Setup: Topology and Virtual Servers	7
3.2	Module 1: iRules Rate Limiting	11
3.3	Module 2: LTM Policy Path-Based Rate Limiting	26
3.4	Module 3: ASM / Advanced WAF DoS Protection	35
3.5	Instructor Answer Key	60

Figure 3.1: General Settings — name, Enforcement Mode = Blocking, Profile Template = Strict. . . .	52
Figure 3.2: Per-class mitigation actions + Strict Mitigation Enforcement Cases.	52
Figure 3.3: Browsers — Browser Verification = Verify Before Access.. . . .	53
Figure 3.4: Signature Enforcement — enforce tooling categories; stage to observe first.	53
Figure 3.5: Whitelist — safe URLs with mitigation and challenges disabled.	53
Figure 3.6: Fake Googlebot from an unverified IP → Malicious Bot, verification failed, denied.. . .	56
Figure 3.7: Same UA from a PTR-verified IP → Trusted Bot, Search Engine, allowed (Alarm). . . .	56
Figure 3.8: Bot Mitigation Settings — Unknown = Rate Limit for 5 transactions per second.	57
Figure 3.9: Over-limit Unknown request → Rate Limit enforced as TCP Reset (Denied)..	57
Figure 3.10: Non-browser automation from kali → classified and mitigated; it never reaches the app.	59
Figure 3.11: superjump’s real browser solves the challenge → Browser (verified); app and assets accepted.	59

Table 3.1: Fixed Window vs. Sliding Window	20
Table 3.2: Lab environment	22
Table 3.3: DoS L7 event lifecycle.	38
Table 3.4: Stress-Based vs. TPS-Based Comparison	40
Table 3.5: Lab environment	44
Table 3.6: Lab environment	48
Table 3.7: Lab environment (UDF blueprint).	50

This lab demonstrates three methods for mitigating Layer 7 Denial of Service (L7DoS) and HTTP rate limiting on F5 BIG-IP, progressing from custom iRule code through declarative LTM policies to full ASM/Advanced WAF behavioral protection.

Module	Method	License Required	Best For
1	iRules	LTM	Custom, granular, per-rule logic
2	LTM Policies	LTM	Declarative path-based enforcement without code
3	ASM / Advanced WAF	ASM	Behavioral, ML-based detection at scale

Lab Environment

```

[Clients]                                     [BIG-IP VE 17.1.0.1 · mgmt 10.1.1.11]
kali      10.1.10.100  (.200 = BaDOS attacker)
win-client 10.1.10.4   (Windows client)
|
| HTTP      vs-lab-irules 10.1.10.55  Module 1 · iRules      -+
+-----> vs-lab-ltm    10.1.10.56  Module 2 · LTM policies -+
          vs-lab-dos    10.1.10.63  Module 3 · DoS profile  -+>
Hackazon_pool -> Hackazon
                  vs-lab-bot    10.1.10.74  Module 3 · Bot Defense  -+
10.1.20.20
                  vs_Hackazon_I 10.1.10.61  Module 3.2 · BaDOS demo  -+

```

Each lab targets its own virtual server (see [Lab Setup: Topology and Virtual Servers](#)), all fronting the same Hackazon_pool. vs_Hackazon_I is the blueprint's prebuilt BaDOS demo VS used by Lab 3.

Important All attack simulation scripts in this lab are for **authorized lab environments only**. Do not execute them against production systems or any system you do not own and have explicit written permission to test.

Prerequisites

- BIG-IP VE 15.1 or later (lab blueprint uses 17.1.0.1)
- ASM / Advanced WAF provisioned (required for Module 3 only)
- TMUI access: `https://10.1.1.11`
- Web Shell access to the lab instances (UDF UI) — see below
- `curl` and `ab` (`apache2-utils`) on the attack client

Accessing the lab environment

Every instance has a **Web Shell** in the UDF UI that opens a **root** shell directly. Use it wherever a lab says “SSH to”, “run on”, or “from” a host (kali, win-client, the Hackazon/LAMP/ELK servers, the BIG-IP) — no SSH client or credentials required. For graphical access — the Windows client, or a browser inside the lab — use the **superjump** host’s **Guacamole** interface (RDP to win-client plus consoles for the other instances).

Management addresses (mgmt subnet 10.1.1.0/24):

Instance	Mgmt IP	Access
BIG-IP	10.1.1.11	TMUI <code>https://10.1.1.11</code> + Web Shell
kali (attacker)	10.1.1.7	Web Shell
win-client (good client)	10.1.1.6	Guacamole (RDP)
Hackazon (Docker backend)	10.1.1.5	Web Shell
ELK / DVGA (Device ID+ / Kibana)	10.1.1.10	Web Shell + Kibana
superjump	10.1.1.8	Guacamole / XRDP

3.1 Lab Setup: Topology and Virtual Servers

Each protection method in this guide is demonstrated on its **own virtual server**, all fronting the same Hackazon backend. Using separate VIPs keeps the methods from interfering with one another and lets you run them side by side against the same attack. (There’s an extra reason to separate the two Bot Defense VIPs if you deploy DoS profiles via AS3 — see the note under the VS map.)

3.1.1 Topology

[Clients]	[BIG-IP VE 17.1.0.1 · mgmt 10.1.1.11]
kali 10.1.10.100 (.200 = BaDOS attacker)	
win-client 10.1.10.4 (Windows client)	
HTTP	vs-lab-irules 10.1.10.55 Module 1 · iRules -+
+----->	vs-lab-ltm 10.1.10.56 Module 2 · LTM policies -+
	vs-lab-dos 10.1.10.63 Module 3 · DoS profile -+>
Hackazon_pool -> Hackazon	
	vs-lab-bot 10.1.10.74 Module 3 · Bot Defense -+
10.1.20.20	
	vs_Hackazon_I 10.1.10.61 Module 3.2 · BaDOS demo -+

3.1.2 Virtual server map

Virtual server	Address	Module / scenarios	What to attach
vs-lab-irules	10.1.10.55:80	Module 1 — iRule rate limiting (1.1–1.5)	The iRule under test (rate-limit-per-ip, rate-limit-per-uri, concurrent-conn-limit, sliding-window-429, custom-l7dos-signature)
vs-lab-ltm	10.1.10.56:80	Module 2 — LTM policies (2.1–2.4)	The LTM policy under test, plus policy-triggered-rate-limit where the scenario uses it
vs-lab-dos	10.1.10.63:80	Module 3 — DoS profile: TPS (3.1), BADoS (3.3), stress (3.2), and the DoS-profile Proactive Bot Defense (3.5, Task 1)	lab_dos_* DoS profile (tps-dos-profile.json / bados-profile.json / bot-defense-profile.json). Lab 3 temporarily swaps this VS to the xff-http profile and attaches the xff-traffic-shaping iRule for bad-actor detection (student steps).
vs-lab-bot	10.1.10.74:80	Module 3 — standalone Bot Defense profile (3.5, Task 2+): verify before/after, per-bot rate limits	security bot-defense profile lab-bot-defense (bot-defense-standalone.conf)
vs_Hackazon_I	10.1.10.61:80	Pre-built by the UDF blueprint — leave as-is	Nothing (unprotected baseline for A/B comparison)

All four lab VIPs point at the blueprint's existing Hackazon_pool, so the application under test is identical across methods.

Note Pool. The lab uses the UDF blueprint's existing Hackazon_pool, which already has a live, monitored Hackazon member — it does **not** create its own pool. (An earlier draft created a pool with member 10.1.20.5:80, but nothing was listening on that address in this blueprint version.) Confirm the pool and its member are up before building:

```
tmsh list ltm pool Hackazon_pool
```

Note Addresses already in use. The UDF blueprint pre-builds several demo virtual servers on the client subnet — .52, .54, .57, .58, .59, .61 (vs_Hackazon_I), .62, .65, and .66 — so those are taken. The lab uses the free addresses .55, .56, .63, and .74 (.78 is spare). .9 is the BIG-IP self IP, not a VIP. Check before building:

```
tmsh list ltm virtual destination
tmsh list ltm virtual-address
```

Leaving vs_Hackazon_I (.61) untouched is handy: it's an unmitigated path to the same Hackazon backend, so you can hit it alongside a lab VIP to compare "no protection" against each mitigation.

3.1.3 Backend CPU state (throughput vs. stress labs)

The Hackazon container is **CPU-limited by default**, which caps how fast it responds — and therefore how many requests per second a closed-loop tool like ab can drive through it. That cuts two opposite

ways:

- **Throughput labs** — Module 1 rate limiting, Module 2 policy rate tiers, Module 3 TPS — must exceed a *per-second* threshold. A CPU-limited backend holds ab below the threshold, so nothing is rejected and the demo looks broken. Give the container full CPU first, from the Hackazon **Web Shell** (10.1.1.5):

```
docker ps
docker update --cpus=0 <hackazon-container>      # 0 = no limit (full speed)
```

- **Stress-based detection** (Module 3 Lab 2) needs the *opposite* — a slow backend — so it deliberately throttles (`docker update --cpus=0.1`) and restores (`--cpus=0`) afterward.
- **Concurrent-connection limiting** (Module 1 Lab 3) is unaffected: it counts open connections, not rate, so backend speed doesn't matter.

3.1.4 Create the virtual servers

Run once on the BIG-IP (mgmt 10.1.1.11). The lab reuses the UDF blueprint's existing Hackazon_pool (which already has a live Hackazon member), so no pool is created here. Adjust the VIP addresses if your deployment differs.

```
# Four virtual servers, all fronting the blueprint's existing Hackazon_pool.
# SNAT automap so the backend returns via the BIG-IP server-side self IP.
tmsh create ltm virtual vs-lab-irules destination 10.1.10.55:80 \
    ip-protocol tcp pool Hackazon_pool \
    profiles add { tcp http } source-address-translation { type automap }

tmsh create ltm virtual vs-lab-ltm destination 10.1.10.56:80 \
    ip-protocol tcp pool Hackazon_pool \
    profiles add { tcp http } source-address-translation { type automap }

tmsh create ltm virtual vs-lab-dos destination 10.1.10.63:80 \
    ip-protocol tcp pool Hackazon_pool \
    profiles add { tcp http } source-address-translation { type automap }

tmsh create ltm virtual vs-lab-bot destination 10.1.10.74:80 \
    ip-protocol tcp pool Hackazon_pool \
    profiles add { tcp http } source-address-translation { type automap }

tmsh save sys config
```

Atomic build + SCF export

To create all four VIPs in a single all-or-nothing **transaction** (nothing is created if any object fails) and export the result as a Single Configuration File, run `scripts/setup/create-lab-vips.sh` on the BIG-IP. It checks that Hackazon_pool exists and that each address is free (clean early abort — no dangling transaction prompt), wraps VIP creation in `create cli transaction / submit cli transaction`, verifies all four VIPs exist, and only then writes `/var/local/scf/l7dos-lab.scf`.

Note An SCF is a **whole-device** configuration snapshot, not just these objects — `tmsh load sys config file l7dos-lab.scf` *replaces* the entire running config (it first backs the old one up to `/var/local/scf/backup.scf`). To pull only the VIPs into an existing box, keep those stanzas in their own file and load with `tmsh load sys config merge file <file>` instead.

Verify:

```
tmsh list ltm virtual one-line | grep vs-lab-
for ip in 55 56 63 74; do
    curl -s -o /dev/null -w "vs .$ip -> ${http_code}\n" http://10.1.10.$ip/
done
```

Each VIP should return the Hackazon application (200) before any mitigation is attached.

Note Attach only the method being demonstrated to its VIP, and leave the others clean. That way a single `curl` from kali (10.1.10.100) shows exactly one mitigation at a time, and you can A/B two methods by changing only the destination port-less address (.55 vs .63, etc.).

Important Keep `vs-lab-dos` and `vs-lab-bot` separate for a clean, isolated demo. There is also a hard conflict if you deploy DoS profiles via AS3: AS3 auto-generates a shadow `f5_appsvcs_<dos-profile>_botDefense` profile, which collides with a standalone Bot Defense profile on the same VS (duplicate-profile error). Building profiles via UI/CLI doesn't create the shadow, but the split is still the cleanest layout.

3.1.5 Pre-staging the traffic scripts

The labs run traffic generators from two hosts. Stage each script where it runs, at the path the labs expect:

Runs from	Script	Stage at
kali (Linux, Web Shell = root)	scripts/attack /http-flood.sh, scripts/setup /baseline-traffic.sh	/home/ec2-user/lab/scripts/...
win-client (Windows Server)	scripts/setup /baseline-traffic.ps1	C:\lab\baseline-traffic.ps1

`win-client` is Windows, so it uses the **PowerShell** `baseline-traffic.ps1` — it cannot run the bash `baseline-traffic.sh`. All `curl`/bash generators run from kali.

kali — open its Web Shell (root) and pull the repo into `/home/ec2-user/lab` so the paths match the labs:

```
git clone https://github.com/snowblind-/BIG-IP-L7DoS-Lab.git /home/ec2-user/lab
chmod +x /home/ec2-user/lab/scripts/attack/*.sh /home/ec2-user/lab/scripts/setup
/*.sh
```

If the instance has no Internet, copy the two `.sh` files in via the superjump **FileBrowser** or the RDP /Guacamole clipboard, keeping the `/home/ec2-user/lab/scripts/...` layout.

win-client — place `baseline-traffic.ps1` at `C:\lab\`. With Internet:

```
New-Item -ItemType Directory -Force C:\lab | Out-Null
Invoke-WebRequest -UseBasicParsing -OutFile C:\lab\baseline-traffic.ps1 `
    https://raw.githubusercontent.com/snowblind-/BIG-IP-L7DoS-Lab/main/scripts/setup
/baseline-traffic.ps1
```

Otherwise copy it over via the Guacamole clipboard / a mapped drive.

3.1.6 Deploying with AS3 (optional, for instructors)

Participants build every object through the **UI or CLI** steps in each lab — no AS3 required. The JSON files under `configs/` (`tps-dos-profile.json`, `bados-profile.json`, `bot-defense-profile.json`, and the Module 2 `*-policy.json` files) are **AS3 declarations** kept as an optional shortcut: an instructor can pre-stage a whole module with one call instead of clicking through. They are reference/pre-provisioning artifacts, not the participant path.

Deploy one (requires the AS3 RPM installed on the BIG-IP):

```
# POST the declaration to the AS3 endpoint
curl -sk -u admin:<password> -H "Content-Type: application/json" \
-X POST https://10.1.1.11/mgmt/shared/appsvcs/declare \
-d @configs/profiles/bados-profile.json
```

Then attach the resulting profile/policy to the relevant `vs-lab-*` VIP as the lab describes. If you deploy DoS profiles this way, mind the AS3 shadow-profile conflict noted above.

3.1.7 Minimal variant

If you want fewer VIPs, the one split you should *not* collapse is `vs-lab-dos` vs `vs-lab-bot` (the conflict above). A workable three-VIP layout merges Modules 1 and 2 onto a single `vs-lab-rate` (iRules and LTM policies coexist fine), keeping `vs-lab-dos` and `vs-lab-bot` distinct.

3.2 Module 1: iRules Rate Limiting

iRules provide flexible, code-level rate limiting that runs in the BIG-IP LTM data plane. Because enforcement happens inside BIG-IP's TMM (Traffic Management Microkernel), rejected requests never reach the pool — protecting both the application servers and the connection table.

Key advantages of iRules for rate limiting:

- No ASM license required — LTM only
- Unlimited granularity: match on IP, URI, headers, cookies, JWT claims, or any combination
- Respond directly with HTTP 429 and standard `Retry-After` headers
- Per-rule thresholds independent of other virtual servers

When to prefer a different method:

- If protected paths change frequently → use LTM Policy + Datagroup (Module 2)
- If behavioral/unknown attack patterns are the concern → use ASM BADoS (Module 3)
- If a no-code GUI-driven approach is required → use LTM Policies (Module 2)

All Module 1 labs attach the iRule under test to **vs-lab-irules** (10.1.10.55). See [Lab Setup: Topology and Virtual Servers](#).

3.2.1 Lab 1: Per-IP HTTP Request Rate Limiting

This lab demonstrates limiting each client IP to a configurable number of HTTP requests per second using the BIG-IP `table` command. Clients that exceed the threshold receive an HTTP **429 Too Many Requests** response directly from BIG-IP — the pool member is never contacted.

Task 1: Upload and Attach the iRule

1. Log in to the BIG-IP TMUI at `https://10.1.1.11`.
2. Navigate to **Local Traffic > iRules > iRule List**.
3. Click **Create**.
4. Set the **Name** to `rate-limit-per-ip`.
5. Paste the contents of `configs/irules/rate-limit-per-ip.tcl` into the **Definition** field.

```
## rate-limit-per-ip.tcl
##
## Limits each client IP to THRESHOLD HTTP requests within WINDOW seconds.
## Clients exceeding the limit receive HTTP 429 until the window resets.
##
## Tuning:
##   THRESHOLD - max requests allowed per window (default: 100)
##   WINDOW    - sliding window in seconds (default: 1)

when RULE_INIT {
    set static::rl_threshold 100
    set static::rl_window 1
}

when HTTP_REQUEST {
    set client [IP::client_addr]
    set key    "rl_ip_[string map {:_ _} $client]"

    set count [table incr $key]

    if { $count == 1 } {
        table set $key $count $static::rl_window $static::rl_window
    }

    if { $count > $static::rl_threshold } {
        HTTP::respond 429 content "Rate limit exceeded. Try again later." \
            "Content-Type" "text/plain" \
            "Retry-After"  $static::rl_window
        return
    }
}
```

6. Click **Finished**.
7. Navigate to **Local Traffic > Virtual Servers** and click **vs-lab-irules**.
8. Select the **Resources** tab.
9. Under **iRules**, click **Manage**.
10. Move **rate-limit-per-ip** from **Available** to **Enabled**.
11. Click **Finished**.

Note The default threshold is **100 requests per second** per client IP. To change it, edit `static::rl_threshold` in the `RULE_INIT` event.

Task 2: Verify Baseline Behavior

1. SSH to the kali attack client (10.1.1.7).
2. Send five sequential requests and confirm all return **200**:

```
for i in $(seq 1 5); do
    curl -so /dev/null -w "%{http_code}\n" http://10.1.10.55/
done
```

Expected output:

```
200
200
200
200
200
```

Task 3: Trigger the Rate Limit

1. **First ensure the Hackazon backend is at full CPU.** The container is CPU-limited by default, which holds ab to ~30 rps — below the 100 req/s limit — so nothing is rejected. On the Hackazon **Web Shell** (10.1.1.5) remove the limit:

```
docker ps                                # note the Hackazon container
docker update --cpus=0 <hackazon-container> # 0 = no CPU limit (full speed)
```

2. From the attack client, run the flood (-l accepts the variable page length so the dynamic body isn't miscounted as failures):

```
ab -n 500 -c 50 -l http://10.1.10.55/
```

3. Observe the response codes in the ab summary. You should see a mix of **200** (within threshold) and **429** (threshold exceeded) responses.
4. Confirm the BIG-IP absorbed the rejects — the pool member sees far fewer requests than ab sent, because the **429** responses never leave the TMM fast path:

- **(kali)** the ab summary already shows the split — 200 (passed) vs 429 (rejected by BIG-IP).
- **(BIG-IP)** tms show ltm virtual vs-lab-irules — compare the client-side vs server-side (pool) counts; the difference is the rejected traffic. (Or TMUI: Statistics > Module Statistics > Local Traffic > Virtual Servers.)

The Hackazon backend runs **Apache** in a Docker container on 10.1.1.5, so there is no /var/log/nginx/ on the host. To watch backend requests directly, open the Hackazon **Web Shell** and tail its access log:

```
tail -20 /var/log/apache2/other_vhosts_access.log
```

If that log lives inside the container rather than on the host, locate it with docker ps then docker exec -it <hackazon-container> tail -20 /var/log/apache2/other_vhosts_access.log.

Interpreting the ab output

A successful run (full-CPU backend) looks like this:

```
Concurrency Level:      50
Time taken for tests:    3.126 seconds
Complete requests:      500
Failed requests:         499
  (Connect: 0, Receive: 0, Length: 499, Exceptions: 0)
Non-2xx responses:      400
Requests per second:    159.96 [#/sec] (mean)
...
Percentage of the requests served within a certain time (ms)
 50%    1
 90%   1497
100%   1762 (longest request)
```

Read it like this:

- **Non-2xx responses: 400** — the rate limit working: 400 requests got **429** (rejected), 100 got **200** (within the 100 req/s window). No Non-2xx line means the limit never tripped — the backend was still CPU-limited and ab stayed under 100 rps; run docker update --cpus=0 on the container and re-run.
- **Failed requests: 499 (Length: 499)** — **not** failures or blocks. ab flags every response whose body length differs from the first; the 200s are the ~64 KB page and the 429s are a tiny message, so lengths vary. -l silences it — all are real 200/429 responses.
- **The percentile split is the proof the 429s never touch the backend:** 50% of requests finished in **1 ms** (the 429s, answered in the TMM fast path) while 90% took ~1.5 s (the 200s that reached

the Hackazon container).

- **159.96 rps** — the achieved rate this run, above the 100/s threshold, which is why the 429s appeared.

Important All **429** responses are generated by BIG-IP in the TMM fast path. They consume no pool member resources — the server never sees rejected requests.

Task 4: Observe the Rate Limit Reset

1. After the flood, wait **two seconds** and send five more requests:

```
sleep 2
for i in $(seq 1 5); do
    curl -so /dev/null -w "%{http_code}\n" http://10.1.10.55/
done
```

Expected output:

```
200
200
200
200
200
```

The table entry TTL has expired, resetting the counter to zero.

Questions

- What would happen to legitimate users behind a shared NAT IP (e.g., a corporate proxy) with 100+ employees browsing simultaneously?
- How would you modify the iRule to issue a warning header at 80% of the threshold before blocking?
- Why is `table incr` used instead of a `table lookup + table set` sequence?

3.2.2 Lab 2: Per-URI Rate Limiting

This lab applies tighter rate limits to specific high-value URI paths — such as authentication endpoints and search — while leaving general site traffic unrestricted. URI-specific limits protect expensive or security-sensitive endpoints without penalizing normal browsing.

Note This lab must exceed a per-second rate threshold. Ensure the Hackazon container is at **full CPU** first — from the Hackazon Web Shell (10.1.1.5), `docker update --cpus=0 <hackazon-container>`. A CPU-limited backend caps ab below the threshold so nothing is rejected. See [Lab Setup: Topology and Virtual Servers \(Backend CPU state\)](#).

Task 1: Upload and Attach the iRule

1. Navigate to **Local Traffic > iRules > iRule List** and click **Create**.
2. Set the **Name** to `rate-limit-per-uri`.
3. Paste the contents of `configs/irules/rate-limit-per-uri.tcl` into the **Definition** field.

```
## rate-limit-per-uri.tcl
##
## Applies tighter per-IP rate limits to specific URI prefixes.
## All other URIs are passed through without restriction.
##
## Tuning:
##   protected_uris - list of URI prefixes to protect
```

```

##   threshold      - max requests per window for protected URIs
##   window         - window size in seconds

when RULE_INIT {
    set static::uri_threshold 20
    set static::uri_window   1

    # Exact match or prefix match - adjust the [string match] below as needed
    set static::protected_uris {
        "/search"
        "/user/login"
    }
}

when HTTP_REQUEST {
    set uri      [HTTP::uri]
    set client   [IP::client_addr]

    set is_protected 0
    foreach prefix $static::protected_uris {
        if { [string match "${prefix}*" $uri] } {
            set is_protected 1
            break
        }
    }

    if { !$is_protected } { return }

    # Sanitize URI for use as a table key
    set uri_key [string map {/ _ ? _ & _} $uri]
    set key "rl_uri_${client}_${uri_key}"

    set count [table incr $key]
    if { $count == 1 } {
        table set $key $count $static::uri_window $static::uri_window
    }

    if { $count > $static::uri_threshold } {
        HTTP::respond 429 content "Rate limit exceeded for this endpoint." \
            "Content-Type" "text/plain" \
            "Retry-After" $static::uri_window
    }
}

```

The default protected URIs and thresholds are:

URI Prefix	Threshold (req/s)	Rationale
/search	20	Expensive DB query
/user/login	20	Credential-stuffing target

4. Click **Finished**.

5. Navigate to **Local Traffic > Virtual Servers**, click **vs-lab-irules**, select the **Resources** tab, click **Manage** under iRules, and add **rate-limit-per-uri** to **Enabled**.

6. Click **Finished**.

Note If **rate-limit-per-ip** from Lab 1 is still attached, remove it before this lab to isolate URI-based behavior.

Task 2: Test an Unprotected Path

1. Flood the home page — this path is **not** in the protected list:

```
ab -n 500 -c 50 http://10.1.10.55/
```

Expected result: all requests return **200**. The home page has no per-URI limit.

Task 3: Test a Protected Path

1. First confirm the path returns **200** on a plain GET (so you are testing the rate limiter, not a backend error) and that the **per-URI** iRule is the one attached:

```
curl -si http://10.1.10.55/search | head -1  
tmsh list ltm virtual vs-lab-irules rules      # expect: rules {  
rate-limit-per-uri }
```

2. Flood the protected search endpoint (backend at full CPU — see the note at the top of the lab):

```
ab -n 400 -c 20 -l http://10.1.10.55/search
```

Expected result: the first ~20 requests (the 1-second window) return **200**; the rest return **429**, shown as Non-2xx responses: <N> in the summary.

3. Confirm the unprotected path is still available in the same second:

```
curl -so /dev/null -w "%{http_code}\n" http://10.1.10.55/
```

Expected result: **200** — the / path is unaffected.

Note Reading the result. Non-2xx responses is your 429 count; ~20 requests getting **200** is the threshold working. The **body size** is the tell for a *real* rate-limit hit — the iRule's 429 is the 38-byte Rate limit exceeded for this endpoint. message. If ab reports a different size (e.g. a 45-byte 404), you are hitting a **backend** response on a path that does not exist, not the limiter. Failed requests: ... (Length: ...) is just the dynamic page varying in size — not real failures (-l silences it).

Task 4: Add a New Protected Path at Runtime

Rather than redeploying the iRule, you can update the `protected_uris` list in the iRule definition.

1. Navigate to **Local Traffic > iRules**, click **rate-limit-per-uri**.
2. Add `/user/login` to `static::protected_uris`:

```
set static::protected_uris {  
  "/search"  
  "/user/login"  
}
```

3. Click **Update**.
4. Verify the new path is now protected (confirm it is 200 first):

```
curl -si http://10.1.10.55/user/login | head -1  
ab -n 400 -c 20 -l http://10.1.10.55/user/login
```

Important Editing an iRule in TMUI causes a brief TMM reload for that rule. For production systems with rapidly changing path lists, use the LTM Policy + Datagroup approach in **Module 2, Lab 3** instead — datagroup edits do not require a rule reload.

Questions

- What happens if a URI has query parameters (e.g., `/search?q=test`)? Does the rate limit treat `/search?q=a` and `/search?q=b` as the same key?
- How would you set different thresholds for different paths in the same iRule?

3.2.3 Lab 3: Concurrent Connection Limiting

This lab limits the number of simultaneous open TCP connections allowed from a single client IP. Unlike request-rate limiting (which counts HTTP requests per second), concurrent connection limiting caps how many connections are open *at the same time* — effective against slow-connection attacks (e.g., Slowloris) and clients that hold connections open.

Note This iRule uses `CLIENT_ACCEPTED` and `CLIENT_CLOSED` events, which fire at the TCP connection level — before any HTTP request is parsed. It works for both HTTP/1.1 and HTTP/2.

Task 1: Upload and Attach the iRule

1. Navigate to **Local Traffic > iRules > iRule List** and click **Create**.
2. Set the **Name** to `concurrent-conn-limit`.
3. Paste the contents of `configs/irules/concurrent-conn-limit.tcl` into the **Definition** field.

```

## concurrent-conn-limit.tcl
##
## Limits simultaneous open connections from a single client IP.
## New connections beyond the limit are reset immediately.
##
## Tuning:
##   MAX_CONNS - max concurrent connections allowed per client IP (default:
20)
##
## The per-IP counter carries a finite idle timeout + absolute lifetime so a
## missed CLIENT_CLOSED (half-open flood, client crash) can't wedge it -
a stale
## count self-heals instead of permanently rejecting new connections.

when RULE_INIT {
    set static::max_conns 20
}

when CLIENT_ACCEPTED {
    set client [IP::client_addr]
    set key    "cc_[string map {:_} $client]"

    set conns [table incr $key]
    # Finite TTLs so a missed CLIENT_CLOSED can't wedge the counter.
    table timeout $key 60
    table lifetime $key 300

    if { $conns > $static::max_conns } {
        reject
        return
    }
}

when CLIENT_CLOSED {
    set client [IP::client_addr]
    set key    "cc_[string map {:_} $client]"

    set current [table lookup $key]
    if { $current ne "" && $current > 0 } {
        table set $key [expr { $current - 1 }] 60 300
    }
}

```

Key parameters:

Variable	Default	Description
static::max_conns	20	Maximum simultaneous connections per client IP

- Click **Finished**.
- Attach `concurrent-conn-limit` to `vs-lab-irules` via the **Resources** tab.

Task 2: Test the Connection Limit

- Confirm the concurrent iRule is the one attached (not a rate-limit iRule from an earlier lab):

```

tmsh list ltm virtual vs-lab-irules rules    # expect: rules {
concurrent-conn-limit }

```

- From the attack client, open 30 simultaneous connections. The home page / is served slowly on the lab backend (~1.5 s), so connections stay open long enough to build past the limit — no artificial slow endpoint needed:

```
for i in $(seq 30); do
    curl -s -o /dev/null -w "%{http_code} " --max-time 30 http://10.1.10.55/ &
done; wait; echo
```

Expected result: about **20** requests print **200** (they got a connection slot) and the rest print **000** — curl's code for "no response", i.e. the connection was **reset** by the iRule once the client passed 20 concurrent.

3. (Optional, while a burst is in flight) check how many connections BIG-IP is tracking for the client — it should not exceed `static::max_conns (20)`:

```
tmsh show sys connection cs-client-addr 10.1.10.100 cs-server-addr 10.1.10.55
| grep -c any
```

Note Reading the results. 000 — and, if you use a plain `&/wait` loop, curl **exit 7** ("couldn't connect") or **exit 56** ("recv failure") — mean the connection was **reset by the limiter**: the mitigation working, not an error. The ~20 that return **200** are the connections allowed under the cap.

You can also see the cap in the **timing**: the rejected connections print 000 almost instantly (reset at connect), while the ~20 allowed 200 responses trickle in over the next ~1.5 s as they finish on the slow backend — the split is visible in *when* the codes appear, not just the counts.

Task 3: Verify Recovery

1. Once the burst above finishes (wait returns, so every connection has closed), send a new request:

```
curl -so /dev/null -w "%{http_code}\n" http://10.1.10.55/
```

Expected result: **200** — as connections close, `CLIENT_CLOSED` decrements the counter, so new connections are accepted again.

Important The `CLIENT_CLOSED` event decrements the counter when a connection closes cleanly. For abruptly dropped connections (e.g., client crash), BIG-IP's TCP half-open timeout will eventually clean up the connection state and fire `CLIENT_CLOSED`. As a safety net, the counter also carries a **60 s idle timeout** and a **300 s hard lifetime**, so a stale count self-heals rather than permanently rejecting new connections.

Reset and teardown

1. The counter self-heals — if the client stops connecting, its entry expires after the idle timeout (60 s). To clear it immediately, drop the client's connections:

```
tmsh delete sys connection cs-client-addr 10.1.10.100
```

2. Restore the virtual server for the next lab by detaching the iRule:

```
tmsh modify ltm virtual vs-lab-irules rules none
```

Note If a burst ever caps far below 20, the counter is wedged from a prior run — wait for the idle timeout, or run the `delete sys connection` above. (The TTLs above prevent this from persisting.)

Questions

- How does this approach differ from BIG-IP's built-in **Connection Rate Limit** on the virtual server?
- What attack type is this *not* effective against, and which iRule lab addresses that scenario

instead?

3.2.4 Lab 4: Sliding Window Rate Limiter with HTTP 429

This lab demonstrates a **sliding window** rate limiter — a more accurate alternative to the fixed-window approach used in Lab 1. A fixed window resets its counter at the boundary of each interval, which allows a burst of $2 \times \text{threshold}$ requests if timed across a window boundary. The sliding window only counts requests within the last N seconds from *now*, eliminating that boundary burst.

Table 3.1. Fixed Window vs. Sliding Window

Property	Fixed Window (Lab 1)	Sliding Window (Lab 4)
Boundary burst	Yes — $2 \times$ threshold possible	No
Memory per client	One counter	List of timestamps
Accuracy	Approximate	Exact
TMM CPU cost	Very low	Low-medium
Best for	High-volume, low-precision	Security-sensitive endpoints

Note This lab must exceed a per-second rate threshold. Ensure the Hackazon container is at **full CPU** first — from the Hackazon Web Shell (10.1.1.5), `docker update --cpus=0 <hackazon-container>`. A CPU-limited backend caps ab below the threshold so nothing is rejected. See [Lab Setup: Topology and Virtual Servers](#) (*Backend CPU state*).

Task 1: Upload and Attach the iRule

1. Navigate to **Local Traffic > iRules > iRule List** and click **Create**.
2. Set the **Name** to `sliding-window-429`.
3. Paste the contents of `configs/irules/sliding-window-429.tcl` into the **Definition** field.

```
## sliding-window-429.tcl
##
## Sliding window rate limiter. Tracks request timestamps per client IP;
## only requests within the last WINDOW seconds count toward the limit.
## Returns HTTP 429 with Retry-After when threshold is exceeded.
##
## Note: More accurate than fixed-window but uses more table memory.
##       Use rate-limit-per-ip.tcl for high-volume, lower-accuracy use cases.
##
## Tuning:
##   THRESHOLD - max requests allowed in any WINDOW-second period (default:
50)
##   WINDOW    - sliding window in seconds (default: 5)

when RULE_INIT {
    set static::sw_threshold 50
    set static::sw_window   5
}

when HTTP_REQUEST {
    set client [IP::client_addr]
    set now    [clock seconds]
    set key    "sw_[string map {:_} $client]"

    # Read existing timestamp list
    set ts_list [table lookup $key]
    if { $ts_list eq "" } { set ts_list {} }

    # Evict timestamps outside the window
```

```

set fresh {}
foreach ts $ts_list {
    if { $ts >= ($now - $static::sw_window) } {
        lappend fresh $ts
    }
}
lappend fresh $now

# Persist updated list
# Idle timeout = window; the entry self-expires once traffic stops (no
# absolute lifetime, which would periodically reset the window
mid-stream).
table set $key $fresh $static::sw_window

if { [llength $fresh] > $static::sw_threshold } {
    HTTP::respond 429 content "Too many requests." \
        "Content-Type" "text/plain" \
        "Retry-After" $static::sw_window \
        "X-RateLimit-Limit" $static::sw_threshold \
        "X-RateLimit-Remaining" "0"
}
}

```

Key parameters:

Variable	Default	Description
static::sw_threshold	50	Max requests in any sw_window-second period
static::sw_window	5	Sliding window size in seconds

4. Click **Finished**.

5. Attach sliding-window-429 to vs-lab-irules.

Task 2: Observe the Retry-After Response

1. Confirm the sliding-window iRule is the one attached (not a rate-limit iRule from an earlier lab):

```

tmsh list ltm virtual vs-lab-irules rules # expect: rules {
sliding-window-429 }

```

2. Flood the virtual server to exceed the threshold (50 requests in any 5-second window). Even the slow backend's ~30 rps clears that (~150 in 5 s):

```

ab -n 200 -c 20 -l http://10.1.10.55/

```

Expected: after the first ~50 requests in the window, the rest return **429** — shown as Non-2xx responses: <N> in the ab summary. No Non-2xx line means either the wrong iRule is attached or the rate stayed under 50/5 s.

3. Use curl -v to inspect the full HTTP 429 response headers:

```

curl -v http://10.1.10.55/ 2>&1 | grep -E "< HTTP|< Retry|< X-Rate"

```

Expected output:

```

< HTTP/1.1 429 Too Many Requests
< Retry-After: 5
< X-RateLimit-Limit: 50
< X-RateLimit-Remaining: 0

```

Task 3: Demonstrate Boundary Burst Prevention

This task shows that a sliding window blocks the double-burst that is possible with a fixed window.

1. Temporarily lower the threshold to **5 requests per 5 seconds** by editing the iRule:

```
set static::sw_threshold 5
set static::sw_window 5
```

2. Send 5 requests in rapid succession:

```
for i in $(seq 1 5); do curl -so /dev/null -w "%{http_code}\n" http://10.1.10.55/; done
```

All 5 return **200**.

3. Immediately (within the same second) send 5 more:

```
for i in $(seq 1 5); do curl -so /dev/null -w "%{http_code}\n" http://10.1.10.55/; done
```

All 5 return **429** — the sliding window sees 10 requests in the last 5 seconds, exceeding the threshold of 5.

Note Interpreting the result. The 200→429 flip *is* the sliding window working: both bursts fall inside the trailing 5-second window, so the second burst is counted *together with* the first — it inherits the running count instead of getting a fresh allowance. That is the boundary-burst defense a **fixed** window lacks: a fixed window resets its counter at each boundary, so an attacker sending 50 requests at 0:04 and 50 more at 0:06 slips 100 through in two seconds (two separate windows). The sliding window sees all 100 within the last 5 s and rejects the overage — you are watching that same effect at 5-request scale.

4. **Teardown:** restore the production threshold you lowered for this task:

```
set static::sw_threshold 50
```

Edit the iRule and click **Update**, so the lab ends at the default 50-per-5 s.

Questions

- At high request volumes (thousands of clients), the timestamp-list approach uses more TMM memory. What is a practical upper bound on clients you would apply this iRule to?
- How would you combine this iRule with the per-URI approach from Lab 2 so that each endpoint has its own sliding window?

3.2.5 Lab 5: Custom L7 DoS Signature — Header-Absence Rate Limiting

This lab builds a **custom L7 DoS signature** that separates legitimate traffic from an attack by a property of the request itself, then mitigates the attack by rate-limiting the offenders to a low TPS.

The signature logic is deliberately simple and effective: a legitimate first-party client (the app's single-page front end, mobile app, or API caller) always sends two application-specific headers. Commodity attack tooling — `curl`, `curl`, `ab`, generic bots — does not. **If either required header is missing, the request is throttled to a low request rate; if both are present, it passes untouched.**

Table 3.2. Lab environment

Role	Address	Notes
Attack client (kali)	10.1.10.100	Sends requests WITHOUT the required headers
Virtual server	10.1.10.55	iRule attached here
Required header 1	X-Client-ID	Sent by legitimate clients
Required header 2	X-Client-Token	Sent by legitimate clients

Why an iRule and not a bot signature

You might expect to build this as a custom *bot signature* in Bot Defense. You cannot, because of two BIG-IP constraints:

Signature type	Limitation
Bot signature	Negation is not permitted. A bot signature can only match content that is <i>present</i> (user-agent, headercontent, uricontent) — it cannot match on a header being <i>absent</i> .
Attack signature	Negation <i>is</i> allowed, but the only actions are block / alarm . Attack signatures cannot rate-limit.

“Header absent → rate-limit TPS” needs both negation and a rate-limit action, so it is implemented as an iRule. iRules run in TMM, need no ASM license, and can match on any combination of IP, URI, headers, or cookies — see the Module 1 overview.

Note The inverse — matching a header/user-agent that *is* present and rate-limiting it — *can* be done natively: write a positive-match custom bot signature, assign it to a category, and set that category’s action to rate-limit with `rate-limit-tps` (see Module 3, Lab 3). Use that when you can enumerate the bad clients; use this iRule when you can only enumerate the *good* ones.

Task 1: Review the signature logic

Open `configs/irules/custom-l7dos-signature.tcl`. The `RULE_INIT` block defines the two required headers and the throttle:

```
set static::sig_req_hdr_1    "X-Client-ID"
set static::sig_req_hdr_2    "X-Client-Token"
set static::sig_suspect_tps  5
set static::sig_window       1
```

On each request the iRule passes traffic that carries **both** headers, and applies a per-source-IP fixed-window counter (5 requests / second by default) to everything else, returning 429 past the cap.

Task 2: Deploy and attach the iRule

1. Create the iRule on the BIG-IP:

```
tmsh create ltm rule custom-l7dos-signature
```

Paste the contents of `custom-l7dos-signature.tcl` (or merge it from a file), then save.

```
## custom-l7dos-signature.tcl
##
## Custom L7 DoS "signature": detects requests that OMIT the application's
## expected client headers and rate-limits them to a low TPS, while requests
## that carry BOTH required headers pass unrestricted.
##
## Why an iRule and not a native signature:
##   * BIG-IP BOT signatures cannot use negation, so they cannot match on a
##     header being ABSENT (F5: "not allowed in bot signatures: negation").
##   * ATTACK signatures allow negation but can only block/alarm - they cannot
##     rate-limit. So "missing headers -> throttle TPS" is expressed here.
##
## Detection: a legitimate first-party client (the app's SPA / mobile app /
## API caller) always sends REQ_HDR_1 and REQ_HDR_2. Commodity attack tools
## (curl, ab, generic bots) do not. Missing either header == suspicious.
##
## Mitigation: suspicious sources are capped at SUSPECT_TPS requests per
## WINDOW
## seconds (per client IP); requests over the cap receive HTTP 429.
##
```

```

## Tuning:
##   sig_req_hdr_1 / sig_req_hdr_2 - headers a legitimate client must send
##   sig_suspect_tps                - request cap per window for header-less
traffic
##   sig_window                    - window length in seconds

when RULE_INIT {
    set static::sig_req_hdr_1  "X-Client-ID"
    set static::sig_req_hdr_2  "X-Client-Token"
    set static::sig_suspect_tps 5
    set static::sig_window      1
}

when HTTP_REQUEST {
    # --- Signature match: do BOTH required headers exist? ---
    if { [HTTP::header exists $static::sig_req_hdr_1]
        && [HTTP::header exists $static::sig_req_hdr_2] } {
        # Request carries the expected client fingerprint - pass unrestricted.
        return
    }

    # --- Mitigation: throttle header-less (suspicious) traffic per source IP
    ---
    set client [IP::client_addr]
    set key     "sig_nohdr_[string map {:_ _} $client]"

    set count [table incr $key]
    if { $count == 1 } {
        # First request of this window - start the countdown timer on
the entry.
        table set $key $count $static::sig_window $static::sig_window
    }

    if { $count > $static::sig_suspect_tps } {
        HTTP::respond 429 content "Rate limited: request is missing required
client headers." \
            "Content-Type"          "text/plain" \
            "Retry-After"           $static::sig_window \
            "X-L7DoS-Signature"     "missing-client-headers" \
            "X-RateLimit-Limit"     $static::sig_suspect_tps \
            "X-RateLimit-Remaining" "0"
        return
    }
}

```

2. Attach it to the lab virtual server:

```
tmssh modify ltm virtual vs-lab-irules rules { custom-l7dos-signature }
```

If other Module 1 rules are attached, ordering does not matter here — this rule returns early for legitimate traffic and responds directly for throttled traffic.

Task 3: Baseline — legitimate client (headers present)

From kali (10.1.10.100), send a fast burst that includes both headers:

```

for i in $(seq 1 30); do
    curl -s -o /dev/null -w "%{http_code}\n" \
        -H "X-Client-ID: web-portal" \
        -H "X-Client-Token: s3cr3t-demo" \
        http://10.1.10.55/
done

```

Expected result: **every request returns 200**. Well-formed clients are never throttled, regardless of rate

— the signature did not match them.

Task 4: Simulate the attack (headers absent)

Now send the same burst *without* the headers — this is what commodity tooling looks like:

```
for i in $(seq 1 30); do
    curl -s -o /dev/null -w "%{http_code}\n" http://10.1.10.55/
done
```

Expected result: the first 5 requests return 200, then the rest return 429 until the 1-second window rolls over. Inspect the mitigation headers on a blocked response:

```
curl -sD - -o /dev/null http://10.1.10.55/ \
    http://10.1.10.55/ http://10.1.10.55/ http://10.1.10.55/ \
    http://10.1.10.55/ http://10.1.10.55/ | grep -i "X-L7DoS\|X-RateLimit\|HTTP/"
```

You should see X-L7DoS-Signature: missing-client-headers and X-RateLimit-Limit: 5 on the throttled responses.

You can also drive the existing flood script (which does **not** send the headers) and watch it collapse to the cap:

```
bash /home/ec2-user/lab/scripts/attack/http-flood.sh http://10.1.10.55 30 50
```

Task 5: Demonstrate effectiveness under load

Run the two side by side to make the contrast obvious:

1. Attacker flood without headers — throttled to ~5 rps, the origin pool sees almost none of it (TMM answers with 429 before the pool is touched).
2. Legitimate client with headers, in parallel — continues to receive 200 at full speed because its requests never match the signature.

This is the core value: the mitigation targets the *attack's* shape, so a flood is neutralized while real users are unaffected — no CAPTCHA, no JS challenge, no IP blacklist to maintain.

Note Interpreting the results. Three signals confirm the signature is doing the work — and where it happens:

- **Discriminator:** header-bearing requests return **200** at any rate (30/30 in Task 3), while header-absent requests return **200** only up to the cap, then **429** (Task 4). Same rate, same URL — the *only* difference is the header, so the rule is keying on header *absence*, not raw rate.
- **Attribution:** each 429 carries X-L7DoS-Signature: missing-client-headers plus X-RateLimit-Limit: 5 / X-RateLimit-Remaining: 0 — so a client (or a packet capture) can see exactly which rule rejected it.
- **Fast-path rejection:** under the http-flood.sh load, ab reports Server Software: BigIP (not Apache), a tiny ~57-byte body, and tens of thousands of rps absorbed at ~3 ms — because HTTP::respond answers in the TMM fast path and the **origin pool is never touched** (a 50,000-request flood reaches the backend as a handful of allowed requests).

These 429s are generated by the **iRule**, not an ASM DoS profile, so they do **not** appear in **Security > Event Logs > DoS > Application Events** (that view is for the Module 3 DoS-profile labs). To log iRule rejections, add a log statement to the iRule or enable an LTM request-logging profile; otherwise the evidence is the ab summary and the response headers above.

Important The two headers are a **shared secret of convenience, not authentication** — an attacker who learns them can add them and bypass the signature. Use this to shed unsophisticated floods cheaply, and layer it with Bot Defense (Module 3) or behavioral detection for adaptive adversaries. For

anything sensitive, validate a signed/rotating token value, not merely the header's presence.

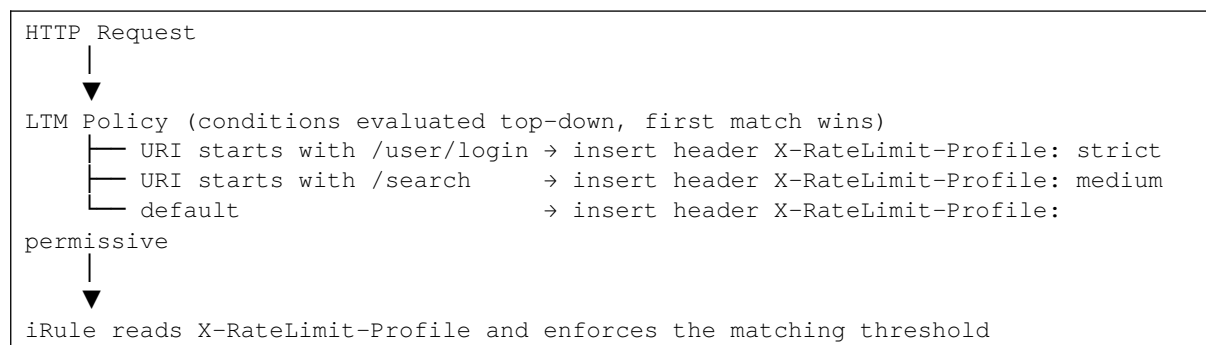
Questions

- The signature keys the counter on client IP. How would you change it to a single global cap for *all* header-less traffic, and when is each better?
- How could you split detection from mitigation using an LTM policy (`http-header ... missing condition`) feeding the Module 2 `policy-triggered-rate-limit.tcl` iRule? What do you gain?
- The current check only tests header *presence*. Sketch the change to also require a valid token *value* in `X-Client-Token` — and note why that moves the logic closer to authentication than DoS mitigation.

3.3 Module 2: LTM Policy Path-Based Rate Limiting

BIG-IP Local Traffic Manager (LTM) policies are declarative rule sets that match HTTP attributes — URI path, method, headers, cookies — and take enforcement actions. For rate limiting they offer a middle path between hand-coded iRules (Module 1) and the full ASM license required for WAF protection (Module 3).

How LTM policies work with rate limiting:



Key advantages:

- Path classification is GUI-manageable without touching iRule code
- A single shared iRule enforces all thresholds — one place to update logic
- Datagroup variant (Lab 3) allows runtime path changes with zero reload
- LTM license only — no ASM required

All Module 2 labs attach the policy under test to **vs-lab-ltm** (10.1.10.56). See [Lab Setup: Topology and Virtual Servers](#).

3.3.1 Lab 1: Path-Based Rate Filter (Native Traffic Shaping)

This lab uses a BIG-IP **traffic class** (rate filter) applied through an LTM policy to cap bandwidth on specific URI paths. No iRule code is required. Enforcement happens in the TMM fast path at the bandwidth level.

Note Rate filters shape **bandwidth (bps)**, not HTTP request rate (req/s). For per-second request limiting, proceed to Lab 2 which combines an LTM policy with an iRule.

Task 1: Create Rate Filter Traffic Classes

1. SSH to the BIG-IP:

```
ssh admin@10.1.1.11
```

2. Create three traffic classes representing different protection tiers:

```
tmsh create ltm traffic-class rate-filter-strict \
    rate 1mbps burst-size 64kb

tmsh create ltm traffic-class rate-filter-medium \
    rate 10mbps burst-size 512kb

tmsh create ltm traffic-class rate-filter-permissive \
    rate 100mbps burst-size 2mb
```

3. Verify the classes were created:

```
tmsh list ltm traffic-class
```

Expected output:

```
ltm traffic-class rate-filter-medium {
    burst-size 512kb
    rate 10mbps
}
ltm traffic-class rate-filter-permissive {
    burst-size 2mb
    rate 100mbps
}
ltm traffic-class rate-filter-strict {
    burst-size 64kb
    rate 1mbps
}
```

Task 2: Create the LTM Policy

1. In the BIG-IP TMUI, navigate to **Local Traffic > Policies > Policy List**.
2. Click **Create**.
3. Set the following:

Field	Value
Name	path-rate-filter-policy
Strategy	first-match
Requires	http

4. Under **Rules**, click **Add Rule**.
5. Create the following rules in order:

Rule 1 — Strict paths

- Name: strict-paths
- Condition: **HTTP URI** | **path** | **begins with** | /user/login
- Action: **Traffic Rate Limiting** | rate-filter-strict

Rule 2 — Medium paths

- Name: medium-paths
- Condition: **HTTP URI** | **path** | **begins with** | /search
- Action: **Traffic Rate Limiting** | rate-filter-medium

Rule 3 — Default

- Name: default-permissive

- Condition: *(none — matches all remaining traffic)*
- Action: **Traffic Rate Limiting** | `rate-filter-permissive`

6. Click **Save Draft**, then **Publish**.

Task 3: Attach the Policy to the Virtual Server

1. Navigate to **Local Traffic > Virtual Servers** and click **vs-lab-ltm**.
2. Select the **Resources** tab.
3. Under **Policies**, click **Manage**.
4. Move `path-rate-filter-policy` from **Available** to **Enabled**.
5. Click **Finished**.

Task 4: Test Bandwidth Shaping

1. From the attack client, download via the **strict** path and observe the capped transfer rate:

```
curl -o /dev/null -w "strict: ${speed_download} bytes/s\n" http://10.1.10.56/user/login
```

The transfer is shaped down toward **1 Mbps**.

2. Download via the **permissive** path:

```
curl -o /dev/null -w "permissive: ${speed_download} bytes/s\n" http://10.1.10.56/
```

Expected: significantly faster (**up to 100 Mbps**).

Note Hackazon's pages are small (tens of KB), so the cap is clearest on a larger asset or a looped/sustained transfer. If your build serves a large static file (a product image or JS bundle), download that via each path for a sharper contrast.

Important Rate filters shape the data stream — they do not drop or reset connections. A client hitting the bandwidth limit will experience slower downloads, not a connection error. For HTTP 429 rejection semantics, use Lab 2 instead.

Questions

- In what scenarios is bandwidth shaping preferable to request-rate limiting?
- How would you adjust burst size to allow short bursts while still enforcing a sustained rate limit?

3.3.2 Lab 2: Policy-Triggered iRule Event Rate Limiting

This lab separates **path classification** (handled by the LTM policy) from **rate enforcement** (handled by a single shared iRule). The policy inserts an `X-RateLimit-Profile` header identifying which tier applies to each request; the iRule reads that header and enforces the corresponding threshold.

This design means:

- Adding a new protected path = policy edit only (no iRule change)
- Changing a rate limit threshold = iRule edit only (no policy change)

Note This lab must exceed a per-second rate threshold. Ensure the Hackazon container is at **full CPU** first — from the Hackazon Web Shell (`10.1.1.5`), `docker update --cpus=0`

<hackazon-container>. A CPU-limited backend caps ab below the threshold so nothing is rejected. See [Lab Setup: Topology and Virtual Servers \(Backend CPU state\)](#).

Task 1: Upload the iRule

1. Navigate to **Local Traffic > iRules > iRule List** and click **Create**.
2. Set the **Name** to `policy-triggered-rate-limit`.
3. Paste the contents of `configs/irules/policy-triggered-rate-limit.tcl` into the **Definition** field.

```
## policy-triggered-rate-limit.tcl
##
## Works with the LTM policy (path-rate-policy.json or
## datagroup-rate-policy.json).
## The policy classifies each request by inserting X-RateLimit-Profile header;
## this iRule reads that header and enforces the corresponding threshold.
##
## For Scenario 3.3 (datagroup), the policy sets profile "datagroup" and this
## iRule performs a second datagroup lookup to get the actual profile name.
##
## Profiles and their thresholds (req/s per client IP):
##   strict      - 10 req/s   (login, checkout, password reset)
##   medium      - 50 req/s   (search, general API)
##   permissive  - 200 req/s  (static content, home page)
##   datagroup   - resolved at runtime from rate_limit_paths datagroup

when RULE_INIT {
    array set static::rl_thresholds {
        strict      10
        medium      50
        permissive  200
    }
    set static::rl_window 1
}

when HTTP_REQUEST {
    set profile [HTTP::header value "X-RateLimit-Profile"]
    HTTP::header remove "X-RateLimit-Profile"

    if { $profile eq "" } { return }

    # Scenario 3.3: resolve profile from datagroup by URI prefix
    if { $profile eq "datagroup" } {
        set uri [HTTP::uri]
        set profile [class match -value $uri starts_with rate_limit_paths]
        if { $profile eq "" } { return }
    }

    set threshold [lindex [array get static::rl_thresholds $profile] 1]
    if { $threshold eq "" } { return }

    set client [IP::client_addr]
    set key     "rl_policy_${profile}_[string map {:_ _} $client]"

    set count [table incr $key]
    if { $count == 1 } {
        table set $key $count $static::rl_window $static::rl_window
    }

    if { $count > $threshold } {
        HTTP::respond 429 content "Rate limit exceeded." \
            "Content-Type"      "text/plain" \

```

```

        "Retry-After"           $static::rl_window \
        "X-RateLimit-Profile"    $profile \
        "X-RateLimit-Limit"      $threshold \
        "X-RateLimit-Remaining" "0"
    }
}

```

Threshold profiles defined in the iRule:

Profile	Threshold (req/s)	Intended paths
strict	10	Login, checkout, password reset
medium	50	Search, general API
permissive	200	Static content, home page

4. Click **Finished**.

Task 2: Create the LTM Policy

1. Navigate to **Local Traffic > Policies > Policy List** and click **Create**.
2. Configure:

Field	Value
Name	path-rate-policy
Strategy	first-match
Requires	http

3. Add the following rules:

Rule 1 — strict-paths

- Condition: **HTTP URI | path | begins with | /user/login**
- Action: **HTTP Header | Insert | Name: X-RateLimit-Profile | Value: strict**

Rule 2 — medium-paths

- Condition: **HTTP URI | path | begins with | /search**
- Action: **HTTP Header | Insert | Name: X-RateLimit-Profile | Value: medium**

Rule 3 — default-permissive

- Condition: *(none)*
- Action: **HTTP Header | Insert | Name: X-RateLimit-Profile | Value: permissive**

4. Click **Save Draft**, then **Publish**.

Task 3: Attach Policy and iRule to the Virtual Server

1. Navigate to **Local Traffic > Virtual Servers**, click **vs-lab-ltm**, and select the **Resources** tab.
2. Under **Policies**, click **Manage** and move **path-rate-policy** to **Enabled**.
3. Under **iRules**, click **Manage** and move **policy-triggered-rate-limit** to **Enabled**.

Important The LTM policy must be evaluated **before** the iRule reads the header. BIG-IP evaluates policies before iRules in the HTTP_REQUEST event by default — no additional ordering configuration is needed.

4. Click **Finished**.

Task 4: Test Per-Path Rate Limiting

(Backend at full CPU — see the note at the top of the lab. Confirm each path is **200** first with `curl -si http://10.1.10.56<path> | head -1`.)

1. Test the **strict** profile on the login page (limit: 10 req/s):

```
ab -n 400 -c 20 -l http://10.1.10.56/user/login
```

You should see **429** (Non-2xx) once the client exceeds 10 req/s — ~10 pass, the rest are rejected.

2. Test the **medium** profile on search (limit: 50 req/s):

```
ab -n 400 -c 20 -l http://10.1.10.56/search
```

Higher throughput is allowed before 429s appear (~50 req/s).

3. Confirm the **permissive** home page is unaffected at moderate rates:

```
ab -n 200 -c 20 -l http://10.1.10.56/
```

Expected result: all **200** at moderate load (well below the 200 req/s permissive limit).

Task 5: Add a New Protected Path Without Editing the iRule

1. Navigate to **Local Traffic > Policies**, click **path-rate-policy**, and click **Edit Draft** (or create a new draft).
2. In **Rule 1 (strict-paths)**, add `/search` to the condition values. Because Rule 1 is evaluated **first** (**first-match**), `/search` is now classified **strict** instead of medium.
3. Click **Save Draft**, then **Publish**.
4. Verify `/search` now trips at the *strict* threshold (10 req/s) — far sooner than the medium 50 req/s it hit in Task 4:

```
ab -n 400 -c 20 -l http://10.1.10.56/search
```

The iRule was not modified — only the policy changed.

Questions

- The iRule removes the `X-RateLimit-Profile` header before forwarding to the pool. Why is this important?
- What happens to requests that match no policy rule and no default rule? How does the iRule handle a missing or empty header?

3.3.3 Lab 3: Dynamic Path Configuration with Datagroups

This lab extends the Lab 2 architecture by storing path-to-profile mappings in a BIG-IP **internal data-group** instead of hard-coded policy conditions. Adding or removing protected paths becomes a single `tmsh` command — no policy edit, no iRule change, and no service interruption.

Note This is the recommended pattern for environments where the list of rate-limited URIs changes frequently (e.g., new API endpoints released weekly) or where network operations staff need to respond to an attack by adding a path limit without a change control window.

Note This lab must exceed a per-second rate threshold. Ensure the Hackazon container is at **full CPU** first — from the Hackazon Web Shell (10.1.1.5), `docker update --cpus=0 <hackazon-container>`. A CPU-limited backend caps `ab` below the threshold so nothing is rejected. See [Lab Setup: Topology and Virtual Servers](#) (*Backend CPU state*).

Task 1: Create the Datagroup

1. SSH to the BIG-IP:

```
ssh admin@10.1.1.11
```

2. Create the datagroup from the lab file:

```
tmsh load sys config merge file \  
    /shared/tmp/rate-limit-paths.dg
```

Alternatively, create it manually:

```
tmsh create ltm data-group internal rate_limit_paths \  
    type string \  
    records add {  
        /search      { data "medium" }  
    }
```

3. Verify the datagroup:

```
tmsh list ltm data-group internal rate_limit_paths
```

Task 2: Update the LTM Policy to Use the Datagroup

1. Navigate to **Local Traffic > Policies**, click **path-rate-policy** from Lab 2, and click **Edit Draft**.
2. Replace all three existing rules with a single new rule:

Rule 1 — datagroup-match

- Condition: **HTTP URI | path | begins with | Datagroup:** /Common /rate_limit_paths
- Action: **HTTP Header | Insert | Name:** X-RateLimit-Profile | **Value:** datagroup

3. Click **Save Draft**, then **Publish**.

Note The action value `datagroup` is a sentinel telling the iRule to perform a second datagroup lookup at request time to resolve the actual profile name (`strict`, `medium`, or `permissive`). This is handled by the `class match` call in `policy-triggered-rate-limit.tcl`.

Task 3: Verify Existing Paths Still Work

1. Test the medium path (backend at full CPU; confirm 200 first with `curl -si http://10.1.10.56/search | head -1`):

```
ab -n 400 -c 20 -l http://10.1.10.56/search
```

Expected: **429** (Non-2xx) once exceeding 50 req/s.

2. Show that a path **not** in the datagroup is unrestricted — it falls to the permissive default:

```
ab -n 400 -c 20 -l http://10.1.10.56/user/login
```

Expected: all **200** (/user/login isn't in `rate_limit_paths` yet).

Task 4: Add a New Path at Runtime Without Any Policy or iRule Change

1. Add /user/login to the datagroup as **strict**:

```
tmsh modify ltm data-group internal rate_limit_paths \  
    records add { /user/login { data "strict" } }
```

2. Immediately re-test it — no reload required. It is now rate-limited where it was unrestricted in

Task 3:

```
ab -n 400 -c 20 -l http://10.1.10.56/user/login
```

Expected: **429** at the strict threshold of 10 req/s.

3. Remove the path when no longer needed:

```
tmsh modify ltm data-group internal rate_limit_paths \
  records delete { /user/login }
```

Task 5: Persist the Datagroup Change

1. Save the running config so the change survives a reboot:

```
tmsh save sys config
```

Important Datagroup modifications take effect immediately in the TMM without a sync or reload. However, they must be saved with `tmsh save sys config` and synchronized to standby units in an HA pair with `tmsh run cm config-sync to-group <device-group>`.

Questions

- How does the class match `-value $uri starts_with rate_limit_paths` call handle URIs that match multiple datagroup entries (e.g., if both a broad `/user` prefix and a specific `/user/login` entry were present)?
- What is the operational trade-off between using a datagroup vs. an external data source (like an iControl REST call) to feed the path list?

3.3.4 Lab 4: Reject Known-Bad Paths with LTM Policy

This lab uses an LTM policy **drop** action to TCP-reset connections to URI paths that should never receive legitimate traffic — WordPress admin panels, exposed config files, PHP management interfaces, and common scanner bait. Because the drop occurs in the LTM policy before the iRule or pool is consulted, it is extremely lightweight.

Note This technique is complementary to rate limiting. Rate limiting defends against high-volume legitimate-looking traffic; path rejection handles requests that are inherently invalid and should never succeed.

Task 1: Create the Reject Policy

1. Navigate to **Local Traffic > Policies > Policy List** and click **Create**.
2. Configure:

Field	Value
Name	reject-bad-paths-policy
Strategy	first-match
Requires	http

3. Add a single rule:

Rule 1 — reject-scanner-bait

- Condition: **HTTP URI | path | begins with** (add each entry):
 - `/wp-admin`
 - `/wp-login`
 - `/.env`

- /.git
- /phpmyadmin
- /admin/
- /xmlrpc.php
- /config.php

- Action: **Forward Traffic** | **Reset**

4. Click **Save Draft**, then **Publish**.

Task 2: Attach the Policy to the Virtual Server

1. Navigate to **Local Traffic > Virtual Servers**, click **vs-lab-ltm**, and select the **Resources** tab.
2. Under **Policies**, click **Manage**.
3. Move **reject-bad-paths-policy** to **Enabled**.

Important If both **path-rate-policy** (Lab 2/3) and **reject-bad-paths-policy** are attached, BIG-IP evaluates them in the order listed. Place the reject policy **first** so scanner bait is dropped before the rate limiter processes it.

4. Click **Finished**.

Task 3: Test Rejection of Known-Bad Paths

1. Attempt to reach a WordPress admin path:

```
curl -v http://10.1.10.56/wp-admin/
```

Expected result: `curl: (56) Recv failure: Connection reset by peer`

2. Attempt to reach an exposed config file:

```
curl -v http://10.1.10.56/.env
```

Expected result: `connection reset — no HTTP response`.

3. Confirm a legitimate path still works:

```
curl -so /dev/null -w "%{http_code}\n" http://10.1.10.56/
```

Expected result: **200**

Task 4: Observe in BIG-IP Logs

1. SSH to the BIG-IP and tail the LTM log while repeating a blocked request in a second terminal:

```
ssh admin@10.1.1.11 tail -f /var/log/ltm
```

Look for entries referencing the reset action from the policy.

2. Alternatively, view policy enforcement in **Statistics > Module Statistics > Local Traffic > Policies** to see hit counts per rule.

Task 5: Add a New Blocked Path at Runtime

1. Edit the draft policy and add **/actuator** (Spring Boot endpoint) to the condition values in **Rule 1**.
2. Publish the draft.
3. Test:

```
curl -v http://10.1.10.56/actuator/env
```

Expected result: connection reset.

Questions

- Why is TCP reset (drop) preferable to responding with HTTP 403 for scanner bait paths?
- How would you combine this lab with Module 3 ASM Bot Defense so that scanners hitting these paths are also added to a block list?

3.4 Module 3: ASM / Advanced WAF DoS Protection

F5 Advanced WAF (formerly ASM) provides three complementary DoS detection modes that operate at the application layer. Unlike iRules and LTM policies (Modules 1 and 2), WAF protection requires an **ASM or Advanced WAF license** but offers behavioral detection that adapts to traffic patterns automatically.

Detection modes:

Mode	Trigger	Action
TPS-based	Request rate exceeds explicit threshold	Block, CAPTCHA, or JS challenge per source/URL
Behavioral (BADoS)	Traffic deviates from learned baseline	Automatic rate shaping and mitigation
Stress-based	Server latency or error rate degrades	Throttle sources causing server stress

Note All three modes can be enabled simultaneously in a single DoS profile. BIG-IP applies the most restrictive applicable mitigation when multiple modes trigger.

Module 3 uses **vs-lab-dos** (10.1.10.63) for the DoS-profile methods (TPS, BADoS, stress, and the DoS-profile Proactive Bot Defense) and **vs-lab-bot** (10.1.10.74) for the standalone Bot Defense profile. See [Lab Setup: Topology and Virtual Servers](#).

3.4.1 Before you begin: enable event logging

Every lab in this module validates through **Security > Event Logs**, which only populate if the virtual server has a security log profile. Attach the existing **L7-DOS_BOT_Logger** profile (it writes DoS-application and Bot Defense events to the local database) to both Module 3 VIPs before starting.

CLI:

```
tmsh modify ltm virtual vs-lab-dos security-log-profiles add { L7-DOS_BOT_Logger }
tmsh modify ltm virtual vs-lab-bot security-log-profiles add { L7-DOS_BOT_Logger }
tmsh save sys config
```

UI: Local Traffic > Virtual Servers > vs-lab-dos > **Security > Policies**, set **Log Profile** to *Enabled*, move **L7-DOS_BOT_Logger** into *Selected*, **Update**; repeat for vs-lab-bot.

If an Event Log stays empty during a lab, a missing log profile on the VS is the usual cause.

3.4.2 Before you begin: the DoS dashboards

Module 3 uses two native DoS dashboards — **no Grafana required**:

- **Security > Overview > DoS** → the **BIG-IP Dashboard** (set the selector to **Behavioral DoS**). A live, per-second view of a protected application: RPS Threshold vs Baseline, **Server Stress**, Concurrent Connections, the **Protected Applications** status (**Calm / Under Attack**), and

a **Detected Attacks** list. Best for watching Behavioral DoS and stress in real time (Labs 2 and 4).

- **Security > Reporting > DoS > Dashboard** → the reporting rollup: the **Attacks** table (Attack ID, severity, vector, trigger, mitigation), **Virtual Servers Health**, **System Health**, and **DoS Attack IDs vs Not attacked**. Toggle **Real Time: ON** for a ~10 s live refresh (otherwise the historical view lags while the rollup catches up). Best for TPS attack events and per-episode/aggregate reporting (Lab 1).

The per-event log is separate: **Security > Event Logs > DoS > Application Events**.

Lab 1: TPS-Based DoS Protection

This lab configures explicit **transactions-per-second (TPS)** thresholds in an ASM DoS profile. When a source IP or URL exceeds the threshold, BIG-IP blocks, challenges, or rate-limits that source until traffic normalizes. TPS-based protection is the simplest WAF DoS mode — it requires no learning period and triggers on known numeric thresholds.

Note This lab must exceed a per-second rate threshold. Ensure the Hackazon container is at **full CPU** first — from the Hackazon Web Shell (10.1.1.5), `docker update --cpus=0 <hackazon-container>`. A CPU-limited backend caps ab below the threshold so nothing is rejected. See [Lab Setup: Topology and Virtual Servers](#) (*Backend CPU state*).

Task 1: Verify ASM is Provisioned

1. SSH to the BIG-IP:

```
ssh admin@10.1.1.11
```

2. Confirm ASM provisioning level is nominal or dedicated:

```
tmsh show sys provision asm
```

Expected output (partial):

```
Sys::Provision
asm    nominal
```

Important If ASM shows *none*, navigate to **System > Resource Provisioning** in the TMUI and set ASM to **Nominal**. The system will require a reboot.

Task 2: Create the TPS-Based DoS Profile

1. In the TMUI, navigate to **Security > DoS Protection > Protection Profiles**.
2. Click **Create**.
3. Set the **Name** to `lab-dos-tps`.
4. Under **Application Security**, open **TPS-based Detection** and set **Operation Mode** to **Blocking** and **Thresholds Mode** to **Manual**.
5. Under **How to detect attackers and which mitigation to use**, enable **By Source IP** — this toggle is required. If it stays off, the thresholds are saved but nothing is ever evaluated (a common “why is nothing detected?” trap). An IP is treated as an attacker if *either* condition is met:

Condition	Value
Relative Threshold — TPS increased by	500% and reached at least 2 TPS
Absolute Threshold — TPS reached	5 TPS

Under **Select mitigation methods to use on the attacking IP's**, tick **Request Blocking** → **Block All**.

Important Set the absolute threshold below the rate one attack client actually reaches on your

backend. A single ab client through the slow lab Hackazon is sampled by the DoS engine at only ~6–20 TPS, so the default absolute threshold of **200** never trips — nothing is detected and the event log stays empty. **5** reliably trips here even when the backend is cold; raise it on a faster backend. Also, the relative “**reached at least**” value must be $\leq$ the absolute threshold (TMOS rejects $min > max$), which is why it is **2** here, not 40.

6. (CLI equivalent) the same settings via tmsh — note the exact 17.5 field names, and that “**ip-rate-limiting enabled**” is the “By Source IP” toggle:

```
tmsh modify security dos profile lab-dos-tps application modify { lab-dos-tps
{ \
    tps-based { \
        ip-rate-limiting enabled \
        ip-maximum-tps 5 \
        ip-minimum-tps 2 \
        ip-tps-increase-rate 500 \
        ip-request-blocking-mode block-all } } }
tmsh save sys config
```

7. (Optional) expand **By URL** and **Site Wide** to set per-URL and whole-site criteria — the **By Source IP** tier is enough for this lab.

8. Set the **Prevention Duration** — this controls how mitigation *ramps up* and *winds down*:

Setting	Value
Escalation Period	30 seconds
De-escalation Period	60 seconds

- **Escalation Period** — how long the system stays at each mitigation step before moving to the next, more aggressive one. Mitigation is applied in steps; if the attack persists past this period, BIG-IP escalates (e.g. from a gentler challenge toward outright blocking).
- **De-escalation Period** — how long the source/URL must stay *below* the thresholds before mitigation is relaxed and finally removed. A longer value prevents **flapping** (mitigation toggling on/off) when traffic hovers near the threshold; too short and a still-active attack resumes the moment mitigation lifts (see the stress-based de-escalation question in Lab 2).

9. Leave **Behavioral & Stress-based Detection** disabled for this lab (those are Labs 2 and 3).

10. Click **Finished**.

Task 3: Attach the DoS Profile and Log Profile to the Virtual Server

A DoS profile is applied at the **virtual server**, not inside an ASM security policy — and it needs a **security log profile**, or nothing reaches the event log and the lab looks like it did nothing.

1. Navigate to **Local Traffic > Virtual Servers**, click **vs-lab-dos**, and open the **Security > Policies** tab.
2. Set **DoS Protection Profile** to **Enabled** and select **lab-dos-tps**.
3. Set **Log Profile** to **Enabled** and move a DoS log profile into **Selected** — **L7-DOS_BOT_Logger** (or **ASM-Bot-DoS-Log-All**; both log DoS/Bot events). This is what populates **Security > Event Logs > DoS > Application Events**.
4. Click **Update**.
Or via CLI (run the two adds on separate lines — chaining them lets a duplicate-profile error abort the log-profile add):

```
tmsh modify ltm virtual vs-lab-dos profiles add { lab-dos-tps }
tmsh modify ltm virtual vs-lab-dos security-log-profiles add {
L7-DOS_BOT_Logger }
```

5. Confirm both are attached:

```
tmsh list ltm virtual vs-lab-dos profiles security-log-profiles
```

Task 4: Generate Attack Traffic and Observe Blocking

1. From the attack client, run a sustained high-rate flood (the backend at full CPU — docker update --cpus=0 — lets ab reach a rate that trips the thresholds):

```
ab -n 50000 -c 100 -t 60 -l http://10.1.10.63/
```

2. While the flood runs, open **Security > Event Logs > DoS > Application Events** (and **Security > Reporting > DoS > Dashboard** with **Real Time: ON** for the live view). Once the source-IP rate crosses the threshold, the attack lifecycle appears — three event types tied together by one **Attack ID**:

Table 3.3. DoS L7 event lifecycle

Event	TPS	What it means
Attack started	20 tps	The profile declared a <i>DOS L7 attack</i> — the measured source-IP rate crossed the detection threshold. Mitigation Source IP-Based Block All is now in effect for this Attack ID.
Suspicious entity	17 tps	The per-source attribution. Entity Type: Source IP / Entity: 10.1.10.100 names the offender. The columns read TPS 17 / Detection Threshold 20 / Mitigate To 13, Threshold Condition: Absolute Manual Threshold — it fired on the absolute ip-maximum-tps you set (20), and rate-limits the entity toward 13. The 20→13 gap is hysteresis that prevents flapping.
Attack ended	11 tps	The rate fell and stayed below the threshold past the de-escalation period (60 s), so the engine cleared the attack — same Attack ID, closing the episode.

(The TPS figures above are from a capture taken with the absolute threshold at **20**; with the lab default of **5** you'll see proportionally smaller numbers — the columns and their meaning are unchanged.)

Read it as **detect (started) → identify + throttle the offending entity (suspicious entity) → recover when traffic subsides (ended)**, all under one Attack ID. "Suspicious entity" is *not* a separate attack — it is the per-source decision *within* the attack, which is why only that row carries an **Entity**. Unlike the Module 1 iRule labs (whose 429s are `HTTP::respond` in the data path), these are real ASM DoS-profile events, so they show up here in **DoS > Application Events**.

Note Each attack episode is one **Attack ID** with an *Attack started* and an *Attack ended* row; a second flood makes a second Attack ID. In the default list view the **Detection Threshold**, **Mitigate To Threshold**, **Threshold Condition**, and **Entity** columns are blank on the started/ended rows — those populate on the **Suspicious entity** rows (the per-source detail). Click an **Attack ID** to drill into an episode, or widen the time filter, to see the suspicious-entity rows and the offending **Entity** (e.g. 10.1.10.100).

Task 5: Review the DoS Reporting Dashboard

The event log is the per-event record; the **DoS Reporting Dashboard** is the aggregate rollup of the same attacks — the view you use to brief on an incident.

1. Navigate to **Security > Reporting > DoS > Dashboard** and set the range to **Last hour** (top-right filter is **HTTP**).
2. Read the panels for your flood:
 - **DoS Attack IDs** (right rail) — one row per episode (e.g. 3183025548) with its transaction count, plus a **Not attacked** row for the legitimate / under-threshold traffic. This is the clean split between attack and normal.

- **Attacks** table — one row per attack: **Severity** (Low here — a modest lab flood), **Vector: Application**, **Trigger: Source IP Vol.**, the **Virtual Server** (/Common/vs-lab-dos), **Mitigation: Blocked**, and **Start/End Time** (one shows *Ongoing* if a flood is still running).
- **Virtual Servers Health** — vs-lab-dos should read **Good** with its latency and client-connection counts: proof the mitigation kept the VS healthy *during* the attack (the whole point).
- **Transaction Origins → Sent by Client** — attributes the attack volume to the client side.

3. Cross-reference: the **Attack ID** shown here is the same ID as in the event log, so you can pivot from the dashboard's aggregate view to the per-event detail and back.

Task 6: Validate Per-Source Mitigation (attacker blocked, others served)

Block All mitigation is scoped to the **offending source IP** — the attacker is cut off while legitimate clients from other IPs keep getting the application. Show both sides during one sustained attack.

1. (**kali — attacker**) Keep a sustained flood running (long enough to test the other source while mitigation is active):

```
ab -n 500000 -c 100 -t 300 -l http://10.1.10.63/
```

2. (**kali — attacker**) From a second kali shell, send a manual request from the same (blocked) source IP:

```
curl -v --max-time 10 http://10.1.10.63/
```

Expected: the connection **hangs and times out** (recv timeout). **Block All drops/resets the attacker's connections — it does not return a block page**, so there is nothing to render. (Reducing the attack rate won't change this; a served page only appears with a *challenge* mitigation — Client-Side Integrity or CAPTCHA — which expect a JS-capable browser, not curl.)

3. (**superjump — legitimate client**) While the attack is **still running**, open the in-browser **Firefox** on superjump (UDF ACCESS > FIREFOX) — a clientless browser session that reaches the VIP from a **different source than the attacker** — and browse to `http://10.1.10.63/`.

Expected: the full **Hackazon** page loads normally. Because its source is not the blocked IP, the per-source mitigation never touches it — the virtual server stays available to everyone except the attacker. *This is the core result of the lab.*

4. (**TMUI**) Cross-reference while both are in flight: in **Security > Event Logs > DoS > Application Events**, open the attack's **Suspicious entity** row — it shows **Entity: 10.1.10.100** and that is the **only** entity listed — the attacker's IP is blocked while the Firefox client (a different source) is not.

Note TPS-based Block All applies to the **source IP** for the mitigation duration, so legitimate users *sharing that IP* (e.g. behind the same NAT) are blocked too, while clients from other IPs are unaffected — exactly what you just demonstrated. For per-user rather than per-IP mitigation, see Behavioral DoS (Lab 3), which targets individual bad actors rather than whole IPs.

Questions

- The TPS Increased By 500% threshold means blocking triggers when the source rate is 5× the **baseline**. What is the baseline, and how is it calculated in TPS-based mode?
- How would you configure different thresholds for different URLs (e.g., stricter for /user/login than for /)?

Lab 2: Stress-Based Detection

Stress-based detection uses **server-side health signals** — response latency and error rates — to identify which clients are contributing to server stress, and throttles those sources proportionally. Unlike TPS-based mode (which blocks sources at a raw request rate), stress-based detection throttles sources

relative to the stress they cause, making it less likely to penalize legitimate heavy users.

Table 3.4. Stress-Based vs. TPS-Based Comparison

Property	TPS-based (Lab 1)	Stress-based (Lab 2)
Trigger	Request rate	Server latency / error rate
Granularity	Per source IP or URL	Per source contributing to stress
Penalizes heavy-but-legitimate users	Possible	Less likely
Requires baseline learning	No	Yes
Effective against slow attacks	Limited	Yes

Task 1: Create a Stress-Based DoS Profile and Switch the Virtual Server to It

Rather than reuse Lab 1's profile, build a **dedicated** stress-based profile from scratch and switch `vs-lab-dos` to it. A virtual server carries **one** DoS profile at a time, so attaching `lab-dos-stress` replaces `lab-dos-tps`; Task 5 puts the shared profile back.

1. (TMUI) Navigate to **Security > DoS Protection > Protection Profiles** and click **Create**. Set **Name** = `lab-dos-stress`.
2. Under **Application Security**, open **Behavioral & Stress-based (D)DoS Detection** and set **Operation Mode** = **Blocking** and **Thresholds Mode** = **Automatic**. Stress-based uses *auto-calculated* thresholds — the system learns normal server stress, so there is no manual latency %% to enter. Leave **TPS-based Detection** disabled: this profile detects by **server stress**, not raw request rate. Configure only the **Stress-based** settings in this panel for this lab — the Behavioral/ML engine (bad-actor behavior, request signatures) is covered in Lab 3.
3. Under **Stress-based Detection and Mitigation** → **By Source IP**, enable a **two-step mitigation ladder** so you can watch mitigation *escalate* as the attack persists: tick **Client Side Integrity Defense** (the gentler first step) and **Request Blocking** → **Block All** (the harder second step). BIG-IP applies the mildest enabled method first and escalates to the next after the Escalation Period if the source keeps driving stress.

Note Client Side Integrity Defense is a JavaScript challenge, so a non-JS client (the curl/ab flood) cannot satisfy it and is dropped at that step; as the flood keeps server stress high, mitigation then escalates to **Block All** — the step-up you observe in Task 3.

4. **Prevention Duration** drives the escalation timing. The stress defaults are long — **Escalation 120 s / De-escalation 7200 s (two hours)** — so mitigation would never relax in lab time. Shorten **both** so the full cycle is visible: **Escalation Period = 15 s** (how long mitigation stays at each step before escalating) and **De-escalation Period = 30 s** (how long stress must stay normal before mitigation relaxes). With these, mitigation steps from Client Side Integrity Defense to Block All about 15 s into a sustained attack, and relaxes ~30 s after it stops.

Note Short periods make the demo snappy but can cause **flapping** (mitigation toggling on/off) when real traffic hovers near the threshold — use longer values in production.

5. Click **Finished**.
(CLI equivalent) — create the profile, add its Application Security container, and arm the By Source IP stress tier. Set **Operation Mode** = **Blocking** and **Thresholds Mode** = **Automatic** in the UI (step above); the tier specifics below are the verified 17.5 keywords. Paste the `modify` as a **single line** — tmsh rejects multi-line brace blocks:

```
tmsh create security dos profile lab-dos-stress
tmsh modify security dos profile lab-dos-stress application add {
lab-dos-stress { } }
tmsh modify security dos profile lab-dos-stress application modify {
lab-dos-stress { stress-based { ip-rate-limiting enabled
ip-client-side-defense enabled ip-request-blocking-mode block-all
escalation-period 15 de-escalation-period 30 } } }
tmsh save sys config
```

Confirm the tier is armed — `ip-rate-limiting enabled` is the By Source IP toggle; if it is disabled nothing is evaluated and the attack is never mitigated:

```
tmsh list security dos profile lab-dos-stress application | grep -A80
"stress-based {" | grep -E "^ *ip-"
```

6. **Switch the virtual server to the new profile.** Navigate to **Local Traffic > Virtual Servers > vs-lab-dos > Security > Policies**. Set **DoS Protection Profile = Enabled** and select `lab-dos-stress` (this replaces `lab-dos-tps`). Ensure **Log Profile = Enabled** with `L7-DOS_BOT_Logger` in **Selected**, then **Update**.

(CLI) — a VS holds one DoS profile, so remove the TPS profile before adding the stress profile; run the adds on separate lines:

```
tmsh modify ltm virtual vs-lab-dos profiles delete { lab-dos-tps }
tmsh modify ltm virtual vs-lab-dos profiles add { lab-dos-stress }
tmsh modify ltm virtual vs-lab-dos security-log-profiles add {
L7-DOS_BOT_Logger }
tmsh list ltm virtual vs-lab-dos profiles security-log-profiles
```

Note Automatic (stress) thresholds need a **learning period** of normal traffic before they are effective — run the baseline (Task 2) first.

Task 2: Establish a Latency Baseline

1. **(kali)** Run the baseline traffic script to allow BIG-IP to learn normal server response times:

```
bash /home/ec2-user/lab/scripts/setup/baseline-traffic.sh http://10.1.10.63
300
```

Allow this to run for at least 5 minutes.

2. In the TMUI, navigate to **Security > Reporting > DoS > Dashboard** (set **Real Time: ON**) and confirm server latency is being tracked under **Virtual Servers Health / System Health**.

Task 3: Simulate a Slow Server Under Attack

1. Open the **Web Shell** for the Hackazon server (UDF UI; mgmt 10.1.1.5, pool member 10.1.20.20) — it drops you in as root. The backend is an **Apache container**, so induce latency by starving the container's CPU from the host (no in-container tooling required):

```
docker ps                                     # note the Hackazon container name
/id
docker update --cpus=0.1 <hackazon-container>
```

With only a fraction of a CPU, the container's response times climb under the attack load in Task 2 — that rising server latency is exactly what stress-based detection keys on.

Note This backend runs **Apache**, not **nginx**, so the older `limit_req_zone` config edit does not apply. If your Docker build doesn't support live `--cpus` updates, peg the CPU from inside instead:

```
docker exec -d <hackazon-container> sh -c 'while ;; do ;; done' # repeat
a few times
```

Either way the goal is the same: make the backend respond slowly under load.

2. From the attack client, generate a high-concurrency load:

```
bash /home/ec2-user/lab/scripts/attack/http-flood.sh http://10.1.10.63 90 80
```

Run it long enough (here 90 s) to outlast the 15 s Escalation Period so the mitigation steps up — then let it stop so you can watch the ~30 s de-escalation.

3. Observe in **Security > Reporting > DoS > Dashboard** (Real Time: ON):

- under **Virtual Servers**, `vs-lab-dos` shows **Server Latency** climbing above baseline with **1** attacking IP (Health stays *Good* — the box throttles the attacker rather than falling over)
- the **Attacks** table lists the episode(s) with **Vector: Application**, **Trigger: Source IP**, **Mitigation: Block** on `/Common/vs-lab-dos`. As the attack persists the severity rises from **Low** to **High**, and the escalated (Block All) episode shows thousands of **Blocked Transactions**; the lower **Low** rows with 0 blocked are the earlier CSID phase / de-escalated episodes

4. **Watch the mitigation escalate.** In **Security > Event Logs > DoS > Application Events**, follow the episode for the attacking source: the first events show **Client Side Integrity Defense** applied, and after ~15 s of continued stress (the Escalation Period) the mitigation **escalates to Block All**. About 30 s after the flood stops and latency returns to normal, the De-escalation Period relaxes mitigation — the step down is logged too.

Note Stress thresholds are automatic and need the Task 2 baseline first; exact escalation timing varies with how fast the backend stress is sampled. If you don't see the step-up, extend the flood and confirm the Escalation Period is 15 s, and that the baseline learning period (Task 2) ran first.

What indicates stress-based detection

Stress-based detection is distinguished by *why* an episode fires, not merely that traffic was mitigated. The tells, all visible in this run:

- **It keys on server health, not request rate.** An episode is declared only when **Server Latency** (or error rate) rises above the learned baseline — the DoS Dashboard's *Virtual Servers* → *Server Latency* for `vs-lab-dos` climbing above normal is the trigger. **TPS-based detection is disabled** in this profile, so request volume alone mitigates nothing; the backend has to actually be stressed.
 - **Cause and effect with the backend.** It is the `--cpus=0.1` throttle that makes the attack register. Restore the CPU (`docker update --cpus=0`) with the *same* flood still running and latency returns to baseline — the episode **de-escalates** even though the clients never slowed down. That dependence on server stress (not on client rate) is the defining signature; the same flood against a healthy, full-CPU backend may not trigger at all.
 - **Automatic, learned thresholds.** Thresholds Mode is **Automatic** — there is no manual TPS number; the system compares live latency to what it learned during the Task 2 baseline. No baseline ⇒ no “normal” to deviate from ⇒ no detection, which is why the learning run is mandatory.
 - **Targeted, proportional mitigation.** Only the sources *contributing to the stress* are acted on, and only *while* the server is stressed: the attacker's IP is throttled then blocked (thousands of **Blocked Transactions**) while a low-rate client on another IP (superjump's Firefox) still gets **200** responses and VS Health stays *Good*. A pure rate limiter acts on rate regardless of whether the server is actually hurting.
 - **In the Attacks table,** the episode reads **Vector: Application** with **Trigger: Source IP** and a *latency-driven* start — contrast a TPS episode, which starts the instant a rate threshold is crossed, stressed or not.
-

Task 4: Verify Proportional Throttling (attacker throttled, others served)

1. **(superjump — legitimate client)** While the attack is still running, open the in-browser **Firefox** on superjump (UDF **ACCESS** > **FIREFOX**, a different source than the attacker) and browse to `http://10.1.10.63/` — the full **Hackazon** page still loads. Stress-based mitigation throttles the sources *causing* the latency, not legitimate clients.
2. **(kali, low-rate)** Optionally quantify it — a low-rate client keeps normal latency and 200 responses while the flood is throttled:

```
for i in $(seq 1 10); do
    curl -so /dev/null -w "Time: %{time_total}s Code: %{http_code}\n" http://10.1.10.63/
    sleep 0.5
done
```

3. **(TMUI)** In **Security > Event Logs > DoS > Application Events**, the episode shows **Detection Mode: DOS L7 attack** on a stress trigger; the **Suspicious entity** rows name the sources contributing most to latency (the attacker). Confirm the Firefox client's source is *not* listed.
4. Compare to Lab 1 (TPS): there, any source over the rate threshold is blocked regardless of server impact; stress-based only mitigates when the server is actually stressed and targets the heaviest contributors.

Task 5: Clean Up

1. In the Hackazon **Web Shell**, remove the artificial stress — restore the container's CPU (0 = no limit), or restart it if you pegged CPU inside:

```
docker update --cpus=0 <hackazon-container>
# or, if you used the busy-loop fallback: docker restart <hackazon-container>
```

2. Verify server response times return to baseline in **Security > Reporting > DoS > Dashboard**.
3. **Switch the virtual server back to the shared profile** for later labs — detach `lab-dos-stress` and reattach `lab-dos-tps` (TMUI: `vs-lab-dos > Security > Policies`, set **DoS Protection Profile** back to `lab-dos-tps`):

```
tmsh modify ltm virtual vs-lab-dos profiles delete { lab-dos-stress }
tmsh modify ltm virtual vs-lab-dos profiles add { lab-dos-tps }
tmsh save sys config
```

(Optional) delete the stress profile if you won't reuse it:

```
tmsh delete security dos profile lab-dos-stress
```

Questions

- A legitimate user running an automated report that makes 200 requests in 10 seconds may cause server latency to rise. How does stress-based detection handle this compared to TPS-based?
- Stress-based detection de-escalates mitigation when latency returns to normal. What is the risk if the de-escalation period is set too short?

Lab 3: Behavioral DoS (BADoS)

Behavioral DoS (BADoS) uses machine learning to model **normal** traffic, then detects and mitigates anomalies automatically — no explicit thresholds. This lab drives the blueprint's ready-made BaDOS demo from kali and validates it in the native **Security > Overview > DoS → Behavioral DoS** dashboard, then inspects the dynamic signatures BADoS generates.

Table 3.5. Lab environment

Role	Access	Notes
Traffic (baseline + attack)	kali — Web Shell (root)	All from kali, split by source IP: baseline binds 10.1.10.100 (good), the attack binds 10.1.10.200 (bad). The demo's XFF_mixed_Attacker_Good iRule on 10.1.10.61 maps .100 → good and .200 → attacker
Validation dashboard	TMUI — Security > Overview > DoS	Native BIG-IP Dashboard → Behavioral DoS (Protected Applications status, Server Stress, RPS threshold/baseline, Detected Attacks) — no Grafana needed
Protected VS	vs_Hackazon_I 10.1.10.61	Blueprint's BaDoS demo VS: Hackazon_BaDOS profile, XFF-http, the XFF_mixed_Attacker_Good iRule, and L7-DOS_BOT_Logger — all pre-wired

Note Run-from key. (kali) = kali attack client via its UDF Web Shell; **(TMUI)** = BIG-IP GUI — the **Security > Overview > DoS → Behavioral DoS** dashboard.

Note This lab reuses the environment's ready BaDOS demo, so the good-vs-bad-actor split, XFF handling, and logging are already configured on vs_Hackazon_I — there is nothing to build first — it is **entirely kali-driven**: the good baseline and the attack both originate from kali, distinguished only by source IP (.100 good vs .200 attacker). You validate from the BIG-IP GUI. (For a build-it-yourself variant on vs-lab-dos, see *Alternative* at the end.)

Task 1: Generate baseline traffic

BaDoS needs a learning period on normal traffic before it can spot anomalies.

1. **(kali)** The Web Shell opens as **root** in root's home, but the prebuilt demo scripts live in /home/ec2-user/ (not /root). cd there before running anything — baseline_menu.sh also reads ./source/useragents_with_bots.txt and ./source/urls.txt by relative path, so it must be launched from that directory:

```
cd /home/ec2-user/      # prompt becomes root@kali:/home/ec2-user#
```

2. **(kali — Web Shell #1)** Start the **increasing** baseline pattern and leave it running in this shell:

```
./baseline_menu.sh
# choose 1 (increasing)
```

3. **(kali — Web Shell #2)** Open a **second** kali Web Shell (cd /home/ec2-user/ again) and start the **alternate** baseline:

```
./baseline_menu.sh
# choose 2 (alternate)
```

Note The UDF Web Shell doesn't pass screen's detach keystroke (Ctrl+a d), so instead of backgrounding with screen, open a **separate Web Shell per long-running stream** — UDF allows several shells to the same host — and leave each running. Stop one with **Ctrl+c** in its own shell.

Both patterns source from 10.1.10.100 with randomised user-agents and URLs, so BaDoS learns a model built from many apparent legitimate clients.

Task 2: Confirm learning

1. (TMUI) Open **Security > Overview > DoS** and set the **Dashboard** selector to **Behavioral DoS**. In **Protected Applications**, select `/Common/vs_Hackazon_I` (profile `Hackazon_BaDOS`) — its **Status** should read **Calm**.
2. Watch **Client HTTP Transactions**: the **Baseline** line settles in and tracks **Incoming Requests** as BAdoS learns normal traffic. Let the baseline establish before attacking — behavioral detection needs a learning period and won't flag an anomaly it has no baseline for.

Task 3: Launch the attack

1. (kali) With the baseline still running, start the attack from a second Web Shell:

```
cd /home/ec2-user/
./AB_DOS.sh
# choose 1 (Attack start - similarity)
```

`AB_DOS.sh` runs looped `ab` floods against `10.1.10.61`, bound to source `10.1.10.200` so the demo's `XFF_mixed_Attacker_Good` iRule stamps them as the attacker cluster. Option **2** (*score*) is a heavier variant you can try instead. The addresses are set at the top of the script (`VS_ADDR / SRC_ADDR*`) — edit them there if your deployment differs.

Note The attack binds to `10.1.10.200`, so kali must have that secondary address configured (it does in the blueprint). Confirm with `ip addr | grep 10.1.10`.

Task 4: Validate mitigation

On the **Security > Overview > DoS → Behavioral DoS** dashboard (`/Common/vs_Hackazon_I`):

1. **Protected Applications → Status** flips from **Calm** to **Under Attack**, and the **Detected Attacks** table gets a row (Attack Id / Start Time / Duration).
2. **Client HTTP Requests & Transactions** shows **Incoming Requests** spike above the **RPS Threshold** while **Server Stress** rises — BAdoS has detected the anomaly.
3. Within a minute or two, **Server Stress** falls back and **Successful Transactions** stay healthy **even though the attack is still running** — BAdoS generated dynamic signatures and is mitigating the bad traffic while legitimate baseline requests keep flowing. The gap between **Incoming Requests** and **Successful / Passthrough** is the dropped attack traffic.
4. (TMUI) **Security > Event Logs > DoS > Application Events** — per-attack detail (anomaly %, mitigated actors, action), populated because `vs_Hackazon_I` carries the `L7-DOS_BOT_Logger` log profile.

Task 5: Examine the signature BAdoS generated

When behavioral detection mitigates an attack it **auto-generates a dynamic signature** describing the attack traffic. Reading it shows *how* BAdoS tells the attack apart from normal traffic.

1. (TMUI) Go to **Security > DoS Protection > Signatures** (the tab next to **Protection Profiles**). Under the **Dynamic** section is an auto-generated signature named `HTTPSig...` tied to your attack — Family **HTTP**, Context `vs_Hackazon_I`, Profile `Hackazon_BaDOS`, and the same **Attack ID** you saw on the dashboard.
2. Read the state columns:
 - **Deployment State: Mitigate** — the signature is actively blocking matching traffic (*vs Detect / Learn*).
 - **Approval State** — dynamic signatures start unapproved; *Manually-approved* means a human vetted it (see Lab 4).
 - **Threshold EPS** (Detection / Mitigation / Dropped / Current) — the events-per-second counters driving it.

3. Expand the row and read the **Predicates String** — the attack's fingerprint, learned automatically. For the AB_DOS.sh flood it looks like:

```
( http.x_forwarded_for_header_exists eq true ) and
( http.referer_header_exists eq true ) and
( http.pragma_header_exists eq true ) and
( http.accept_encoding_header_exists eq true ) and
( http.accept contains application ) and
( http.cache_control_header_exists eq true ) and
( http.headers_count eq 11 ) and
( http.unknown_header_exists eq true ) and
( http.hdrorder hashes-to 11 ) and
( http.cache_control hashes-to 14 ) and
( http.referer hashes-like http://10.0.2.1/none.html )
```

Each predicate is a trait BADOs found common to the attack but **not** to the learned baseline — the specific header set, the header *count* (11) and *order* hash, and the tell-tale *Referer*: `http://10.0.2.1/none.html` the attack script sends. Together they match the attack precisely while leaving normal traffic alone — which is why legitimate clients kept being served during the attack.

Note Why predicate mitigation beats a rate limit here. The signature filters on the *request's shape*, not its source — so it mitigates the attack on different terms than the per-IP and TPS limits in Labs 1 and 3:

- **Source-independent.** It drops anything matching the attack fingerprint regardless of IP, so an attacker rotating thousands of IPs — or hiding behind the same NAT/CDN as real users — is still caught. A per-IP limit only sees source IP, so it misses a distributed/low-and-slow attacker that stays under the threshold.
- **No collateral damage.** Because the predicates match traits common to the attack and *absent* from the learned baseline, legitimate users keep being served even from the same IPs — the opposite of per-IP Block All, which also blocks innocent users sharing a NATed address (the shared-NAT problem from Module 1 Lab 1). That's the 47.8k blocked transactions with the server-side graph staying flat.
- **Automatic and adaptive.** BADOs derives the predicate set from live traffic in seconds — you don't have to know or hand-tune the right threshold in advance (contrast Lab 1, where you had to find a TPS number that fit the backend).

The trade-off: predicate mitigation is sharper but needs the attack to have a learnable, distinct shape and a good baseline; a rate limit is blunt but simple, deterministic, and needs no learning. In production you layer them — behavioral/predicate for precision, rate limits as a coarse back-stop.

4. Expand **Most Recent Attacks: Accuracy 100%** and a non-zero **Current EPS** mean the signature is matching live attack traffic; **Detection / Mitigation Threshold EPS** show the rates at which it acts.
5. (optional) With the signature selected, use **Make Persistent** to keep it beyond this attack (hand-off to [Lab 4: Custom Persistent DoS Signatures](#)), **Set Deployment State**, or **Set Threshold Mode**.

Teardown

1. (**kali**) Stop the attack: press **Ctrl+c** to break the flood loop, then in the AB_DOS.sh menu choose **3 (Attack end)** to `killall ab`, then **4 (Quit)**.
2. (**kali**) Stop the baseline streams — press **Ctrl+c** in each of the Web Shells running `baseline_menu.sh` (or, from any shell):

```
pkill -f baseline_menu.sh
```

Alternative: build-it-yourself on vs-lab-dos

To construct an equivalent demo from scratch on the lab VIP instead of the prebuilt one, use vs-lab-dos with the shared lab-dos-tps profile (enable **Behavioral & Stress-based Detection** on it), swap it to the XFF-http profile, and attach configs/irules/xff-traffic-shaping.tcl — then generate a good baseline and an attack from sources the iRule classifies differently, and validate in the TMUI (**Security > DoS Protection > DoS Overview** → Bad Actors). Set the iRule's good /attack source lists to match whatever hosts you drive traffic from (e.g. win-client + the C:\lab\baseline-traffic.ps1 baseline, or kali's two addresses as the prebuilt demo does).

The xff-traffic-shaping iRule:

```
## xff-traffic-shaping.tcl
##
## Lab prep for Behavioral DoS (BADoS) bad-actor detection.
##
## A lab doesn't have dozens of distinct client IPs, so BADoS can't show its key
## behaviour: singling out the bad actor(s) among many legitimate users. This
## iRule synthesises that by inserting an X-Forwarded-For header per request:
## * good sources -> a fully random XFF (looks like many distinct users)
## * attack sources -> a narrow XFF range (looks like a small repeat-offender
##                      cluster that bad-actor detection can greylist)
##
## REQUIRES: the virtual server's HTTP profile must have "Accept XFF" enabled
## (profile XFF-http), or the DoS profile ignores the injected header and keys on
## the real TCP source. When the source is learned from an L7 header, bad-actor
## mitigation is applied as an HTTP rate-limit rather than a TCP-based one.
##
## Attach to vs-lab-dos for the Module 3 Lab 3 bad-actor demo ONLY, then detach
## it - leaving it on would feed synthetic XFF into the other vs-lab-dos labs.
##
## Tuning (RULE_INIT):
## xff_good_sources - real client IPs treated as legitimate (win-client)
## xff_attack_sources - real client IPs treated as the attacker (kali)
## xff_attack_prefix - the /24 the attacker "hides" in
## xff_attack_hosts - how many distinct attacker IPs to spread across
## xff_debug - 1 to log each good-source XFF to /var/log/ltn

when RULE_INIT {
    set static::xff_good_sources { 10.1.10.4 } ;# win-client
    set static::xff_attack_sources { 10.1.10.100 10.1.10.200 } ;# kali (.100
primary, .200 secondary)
    set static::xff_attack_prefix "132.173.99"
    set static::xff_attack_hosts 25
    set static::xff_debug 0
}

when HTTP_REQUEST {
    set src [IP::client_addr]

    # Good traffic -> fully random XFF (simulates many distinct legitimate users)
    if { [lsearch -exact $static::xff_good_sources $src] >= 0 } {
        set xff "[expr {int(rand()*100)}].[expr {int(rand()*100)}].[expr
{int(rand()*100)}].[expr {int(rand()*100)}]"
        HTTP::header insert X-Forwarded-For $xff
        if { $static::xff_debug } { log local0.info "xff-shaping good $xff from
$src" }
        return
    }

    # Attack traffic -> narrow XFF range (simulates a small repeat-offender
cluster)
    if { [lsearch -exact $static::xff_attack_sources $src] >= 0 } {
        set xff "$static::xff_attack_prefix.[expr
```

```
{int(rand()*$static::xff_attack_hosts)}}"
    HTTP::header insert X-Forwarded-For $xff
}
}
```

Questions

- The dashboard shows **Server Stress** recover *while the attack is still running*. What did BADoS do between “under attack” and “healthy again”, and why is that different from a static rate limit?
- The attacker is blacklisted via **dynamic signatures**. How is that different from the **bad-actor greylist**, and when would each be the mitigation you see?
- A promoted dynamic signature (Task 5) becomes persistent. What do you gain by persisting it, and what’s the risk of persisting one generated during a noisy, mixed attack?

Lab 4: Custom Persistent DoS Signatures

The **Security > DoS Protection > Signatures** screen holds two kinds of DoS signature. This lab builds and mitigates with the *persistent* kind — including one you author yourself in the GUI.

Kind	Description
Dynamic	Auto-generated by the Behavioral DoS (BADoS) engine when it detects server stress and traffic that deviates from the learned baseline. Transient — they age out, and the list is capped (1024 HTTP per object).
Persistent	Permanent. Either a dynamic signature promoted with Make Persistent , or a user-defined signature you author from scratch with your own predicates. Can be shared across protected objects.

Unlike a Bot signature (Module 1, Lab 5 covers why those can’t negate), a user-defined *persistent DoS* signature is hand-authored: you pick the predicates — `metric / operator / arguments` — that match the attack. Notably, `http.<name>_header_exists eq false` lets a signature match on a header being **absent** — the DoS-profile-native counterpart to the Module 1 header-absence iRule.

Table 3.6. Lab environment

Role	Address	Notes
Attack client (kali)	10.1.10.100	Generates the flood
Virtual server	vs-lab-dos 10.1.10.63	Runs the shared lab-dos-tps profile with Request Signatures enabled
CLI supplement	configs/profiles /dos-persistent-signatures	Merge-loadable equivalent + promote /approve commands

Note Persistent signatures are only evaluated when the DoS profile on the VS has application-layer **request-signature detection** enabled. This lab reuses the shared `lab-dos-tps` profile on `vs-lab-dos`: open it, go to **Behavioral & Stress-based (D)DoS Detection → By Bad Actors Behavior / Signatures**, and confirm **Request signatures detection** is ticked. If you also tick **Use approved signatures only**, *only* signatures whose Approval State is Manually-approved will mitigate — dynamic or persistent alike.

HTTP-family rules to know

The Add Signature form and TMOS enforce these for any `http` DoS signature:

Field	Allowed for HTTP
Type	Persistent (you cannot <i>create</i> a dynamic signature)
Deployment State	Learn Only or Mitigate
Threshold Mode	Manual (detection/mitigation thresholds are 0 — HTTP signatures match on predicates, not events-per-second)

Approval State	Approved (Manually-approved) to mitigate under “use approved only”
----------------	--

Task 1: Author a custom persistent signature (GUI)

1. Go to **Security > DoS Protection > Signatures** and select the **Persistent** tab.
2. Click **Add Signature**. Set **Family** = HTTP and name it `lab_sig_scripted_flood`.
3. Build the **Predicates List** — click **Add**, choose a metric, an operator, and a value; repeat to AND several conditions together. For a scripted flood, a good starting set is:

Metric	Operator	Value
<code>http.user_agent_header_exists</code>	<code>eq</code>	<code>false</code>
<code>http.request.method</code>	<code>eq</code>	<code>GET</code>
<code>http.uri_len</code>	<code>between</code>	<code>0-15</code>

The builder shows the metrics your version supports. Common HTTP ones: the `*_header_exists` booleans (user_agent, referer, accept, host, pragma, cache_control, x_forwarded_for, unknown, or any custom header), plus `http.request.method`, `http.uri_len`, `http.uri_parameters`, `http.uri_file`, `http.accept`, `http.headers_count`, `http.referer`. Operators include `eq`, `contains`, `between`, `hashes-to`, and `hashes-like`.

4. Save. To version-control the exact definition, capture it from the CLI and commit it into `dos-persistent-signature.conf`:

```
tmsh list security dos dos-signature lab_sig_scripted_flood
```

Task 2: Approve and activate

1. Open the signature’s properties. Set **State** = Mitigate and tick **Approved** (Approval State → Manually-approved). Tick **Shareable** if other VSs should use it.
2. (Optional, stricter) On the shared `lab-dos-tps` profile tick **Use approved signatures only**, so nothing mitigates until you’ve approved it — the safe way to roll signatures out in production.

Task 3: Test the custom signature

1. From kali (10.1.10.100), send traffic that matches the predicates — a plain header-less curl (no User-Agent, method GET, short URI):

```
for i in $(seq 1 200); do
    curl -s -o /dev/null -w "%{http_code}\n" -H "User-Agent;" http://10.1.10.63/
done
```

Matching requests are mitigated per the profile once the attack threshold is met. (`-H "User-Agent;"` strips curl’s default UA so the `user_agent_header_exists eq false` predicate fires.)

2. Confirm the hit and its state:

```
tmsh show security dos dos-signature lab_sig_scripted_flood
```

Look for a non-zero **Attack Status** / drops.

3. Send a browser-like request (with a normal User-Agent) and confirm it passes — the signature targets the attack’s shape, not all traffic.

Task 4: Promote a dynamic (BADOs) signature to persistent

The other route is to let BADOs find the attack, then keep the signature it generated.

1. Run the behavioral attack from Lab 3 so BADOs generates dynamic signatures:

```
bash /home/ec2-user/lab/scripts/attack/http-flood.sh http://10.1.10.63 60 100
```

2. On the **Dynamic** tab, review the generated signatures (each expands to show its predicate list). Pick an effective one, select it, and click **Make Persistent**, then mark it **Approved**. CLI equivalent:

```
tmsh list security dos dos-signature { type dynamic }
tmsh modify security dos dos-signature <HTTPSig...> \
    type persistent shareability-state fully-shareable \
    approval-state manually-approved state mitigate
```

A promoted signature keeps `origin dynamic-bdos` (vs user-defined for hand-authored), so you can always tell how each one was created.

3. Re-run the attack and confirm the now-persistent signature mitigates immediately, without waiting for BADoS to re-learn.

Task 5: Share the signature to another VS

A **Shareable** persistent signature applies to any other protected object whose DoS profile has signatures enabled — bind the BADoS profile to a second VS and confirm the same signature mitigates there. A non-shareable signature only applies to the context that created it.

Important A persistent signature is a static match — an attacker who changes the matched attribute (adds a User-Agent, lengthens the URI) evades it. Use persistent signatures to lock in mitigation for a *known* attack shape, and keep BADoS running for the adaptive, unknown ones. The two are complementary.

Questions

- When would you set a signature to Learn Only or Detect Only rather than Mitigate — and how does “Use Approved Signatures Only” make rolling one out safer?
- Your signature ANDs three predicates. What breaks if you keep only `http.user_agent_header_exists eq false` — too broad, too narrow, and which legitimate clients might it catch?
- A promoted dynamic signature keeps `origin dynamic-bdos`; a hand-authored one is `user-defined`. Why does that provenance matter when auditing which signatures are enforcing, and who changed them?

Lab 5: Proactive Bot Defense

Proactive Bot Defense injects a JavaScript challenge that legitimate browsers execute transparently to earn a session cookie. Tools that cannot run JavaScript — scripts, `curl`, most bots — never earn the cookie and are mitigated.

Table 3.7. Lab environment (UDF blueprint)

Role	Address / access	Notes
BIG-IP	10.1.1.11 (TMUI / Web Shell)	17.1.0.1 (BYOL)
Attack client (kali)	10.1.10.100 — Web Shell	Runs the <code>curl</code> tests; no JS engine, so it fails the challenge
Legit client (win-client)	10.1.10.4 — Guacamole RDP	Real browser — solves the JS challenge and passes
Bot Defense VS	vs-lab-bot — 10.1.10.74	Carries the standalone Bot Defense profile; the target the tests hit
Backend	Hackazon_pool (member 10.1.20.20)	Blueprint’s existing pool + live Hackazon member

Note Run-from key. Every test/validation below is tagged with where to run it: **(kali)** = kali attack client via its UDF **Web Shell** (`curl`); **(win-client)** = the Windows client via **superjump Guacamole RDP** (a real browser); **(TMUI)** = the BIG-IP GUI, reachable from any browser (win-client or superjump). Configuration steps run on the BIG-IP (TMUI or Web Shell).

This lab uses a **standalone Bot Defense profile** (`security bot-defense profile`) attached to `vs-lab-bot`. Unlike the bot-signature sub-section bundled inside a DoS profile — which can only *block or report* a signature category — the standalone profile provides the full feature set this lab demonstrates:

- per-class actions (Browser / Trusted Bot / Untrusted Bot / Malicious Bot ...)
- **verify-before vs. verify-after** access (when the JS challenge is issued)
- **search-engine (FCrDNS) verification** and **Unknown-class rate limiting**
- custom bot signatures and granular allowlists

(`configs/profiles/bot-defense-standalone.conf` is the reference config.)

Before vs. After Access Verification

The single most important Bot Defense setting is *when* the JavaScript challenge is issued relative to the application seeing the request.

Verification action	Behaviour	Use when
<code>browser-verify-before-access</code>	Challenge served first . The origin app never sees the request until the browser solves the challenge and presents a valid cookie.	Highest protection; login pages, checkout.
<code>browser-verify-after-access-blocking</code>	Request is passed to the app ; JS is injected into the response. If the next request fails the challenge it is mitigated.	Latency-sensitive paths where a one-request exposure is acceptable.
<code>browser-verify-after-access-detection</code>	Same as above but report-only — failures are logged, never blocked.	Tuning / staging before enforcing.
<code>browser-challenge-free-verification</code>	Header inspection only, no JS challenge .	APIs and clients that cannot run JavaScript.

The cookie flow is the same in every case: solve once, present the cookie on subsequent requests, and you are not re-challenged for the session.

Task 1: Create the Standalone Bot Defense Profile

Build the profile in the 17.5 UI so every setting is explicit; the `tmsh merge` in the last note is the instructor fast-path that produces the same result. All steps are **(TMUI)** unless marked CLI.

1. **General Settings.** Navigate to **Security > Bot Defense > Bot Profiles** and click **Create**. Set:

- **Profile Name:** `lab-bot-defense`
- **Enforcement Mode:** **Blocking**
- **Profile Template:** **Strict** — sets strong defaults you then tune (verify-before-access; block untrusted / suspicious / malicious / unknown; DoS Attack Mitigation Mode enabled). *Relaxed* (challenge-free) and *Balanced* (verify-after-access) are the gentler alternatives.
- Leave **Signature Staging upon Update Disabled**, **Enforcement Readiness Period** 7 days, **Redirect to Pool** None, and **Response and Blocking Pages** on **Default**.

Bot Profile General Settings

Figure 3.1. General Settings — name, Enforcement Mode = Blocking, Profile Template = Strict.

(CLI equivalent):

```
tmsh create security bot-defense profile lab-bot-defense template strict
enforcement-mode blocking description "L7DoS Lab - standalone bot defense"
```

2. **Bot Mitigation Settings** — the action applied *after* a client is classified. With **Strict** the defaults are Trusted=Alarm, Untrusted=Block, Suspicious Browser=Block, Malicious=Block, Unknown=Block. Each is a dropdown (**None / Alarm / CAPTCHA / Block / Honeypot Page / Redirect to Pool / TCP Reset**). Tune two classes so the gentler actions are demonstrated:

Class	Action	Why
Trusted Bot	Alarm	Verified good bots (search engines) — log, don't block.
Untrusted Bot	CAPTCHA	Non-malicious tools/crawlers — challenge rather than hard-block.
Suspicious Browser	CAPTCHA	Let a real-but-odd browser prove itself.
Malicious Bot	Block	DoS tools / scanners — block outright.
Unknown	Block	Unclassified non-browser clients.

Leave **DoS Attack Mitigation Mode = Enabled** (Strict default): during a DoS attack it overrides the per-class actions to Browser=Verify-Before-Access, Trusted=Alarm, everything else=Block (it requires a DoS protection profile to be enabled).

Bot Mitigation Settings per-class actions

Figure 3.2. Per-class mitigation actions + Strict Mitigation Enforcement Cases.

(CLI equivalent):

```
tmsh modify security bot-defense profile lab-bot-defense class-overrides
replace-all-with { "Trusted Bot" { mitigation { action alarm } } "Untrusted
Bot" { mitigation { action captcha } } "Suspicious Browser" { mitigation {
action captcha } } "Malicious Bot" { mitigation { action block } } }
```

Note Paste each tmsh brace command as a **single line**. tmsh does not reliably accept backslash-continued multi-line brace blocks pasted into the shell — it errors with “*properties must be enclosed in braces.*”

3. **Browsers** — when/how the JavaScript challenge is issued:

- **Browser Access: Allow**
- **Browser Verification: Verify Before Access** — proactive: the challenge is served first and the app only sees the request after the browser solves it. This is the Strict default and the behaviour Tasks 2–3 demonstrate; switch to **Verify After Access (Blocking/Detection)** to show the “after” variant.
- **Device ID Mode: Generate Before Access** (Strict default)
- Leave **Single Page Application** and **Cross Domain Requests** at the template defaults unless the app needs them.

Browsers - Browser Verification before access

Figure 3.3. Browsers — Browser Verification = Verify Before Access.

4. **Signature Enforcement** — the installed bot signatures (2,600+), grouped by **Bot Class** and **Bot Category**, each **Staged** or **Enforced**. Filter by category and review: keep **Search Engine / Site Monitor** benign; select the tooling categories — **HTTP Library**, **DOS Tool**, **Vulnerability Scanner** — and click **Enforce** (use **Stage** to observe without blocking for the readiness period first).

Signature Enforcement list

Figure 3.4. Signature Enforcement — enforce tooling categories; stage to observe first.

5. **Whitelist** — paths/sources exempt from mitigation and challenges (consulted **first**). Click **Create** and add known-safe assets with **Mitigation** and **Challenges** both **off**:

Source	URL	Mitigation	Challenges
Any	/favicon.ico	off	off
Any	/health	off	off

Do **not** whitelist the whole client subnet — it contains kali (10.1.10.100) and would exempt the attacker.

Whitelist entries

Figure 3.5. Whitelist — safe URLs with mitigation and challenges disabled.

6. **Save** the profile, then **bind** it to the virtual server — **Local Traffic > Virtual Servers > vs-lab-bot > Security > Policies**, set **Bot Defense Profile** = lab-bot-defense > **Update**. (CLI):

```
tmsh modify ltm virtual vs-lab-bot profiles add { lab-bot-defense }
tmsh list ltm virtual vs-lab-bot profiles
```

7. **Attach the security log profile** — Bot Defense events only reach **Security > Event Logs > Bot Defense** if vs-lab-bot carries a log profile. In **Security > Policies** set **Log Profile** = **Enabled** and move **L7-DOS_BOT_Logger** into **Selected > Update**. (CLI) (a duplicate-profile error just means it is already attached):

```
tmsh modify ltm virtual vs-lab-bot security-log-profiles add {
L7-DOS_BOT_Logger }
tmsh list ltm virtual vs-lab-bot profiles security-log-profiles
```

If a Bot Defense event log stays empty during this lab, a missing log profile on the VS is the first thing to check (also the module's [Module 3: ASM / Advanced WAF DoS Protection Before you begin](#) step).

Note Instructor fast-path. The whole profile is in configs/profiles/bot-defense-standalone.conf — merge it instead of clicking through, then bind:

```
tmsh load sys config merge file /var/tmp/bot-defense-standalone.conf
tmsh modify ltm virtual vs-lab-bot profiles add { lab-bot-defense }
```

The .conf also caps the **Trusted Bot** class with rate-limit-tps 5 — a throughput cap set on the class mitigation *alongside* its action (there is no rate-limit action in 17.5). The UI Bot Mitigation

dropdown sets the action only; the `rate-limit-tps` cap is a CLI refinement (see Task 5).

Note Mitigation precedence (order BIG-IP consults): Whitelist → DoS Attack Mitigation Mode → Microservices → Verified API Access → per-class profile mitigation. A whitelisted path is never challenged, and during a DoS attack the DoS-mode settings override the per-class actions.

Task 2: Demonstrate the Challenge — BEFORE access

With `browser-verify-before-access` (the value shipped in the config), compare a script against a real browser:

1. **(kali)** From the kali Web Shell, request the page with `curl`:

```
curl -sv http://10.1.10.74/ 2>&1 | head -40
```

Expected: the response body is the **JavaScript challenge**, not the app. The origin server never received the request — BIG-IP answered first.

2. **(win-client)** Open `http://10.1.10.74/` in a real browser (Guacamole RDP to the Windows client). It briefly renders the challenge, runs the JS, receives a `TS<...>` cookie, and is forwarded to the app in under two seconds. Reloading does not re-challenge (cookie is presented).
3. **(win-client)** Inspect the cookie in the browser's **DevTools > Application > Cookies** — note the TS prefixed bot-defense cookie.

This kali-vs-win-client contrast *is* the demonstration: identical request, blocked for the script, transparent for the browser.

Task 3: Demonstrate the Challenge — AFTER access

Configuration (BIG-IP). Switch the Browser class to after-access and compare:

```
tmsh modify security bot-defense profile lab-bot-defense class-overrides modify {  
  Browser { verification { action browser-verify-after-access-blocking } } }
```

1. **(kali)** Repeat the `curl` from Task 2. This time the **application response is returned** with the verification JavaScript injected into it — the app *did* see the first request. `curl` cannot solve the JS, so the **next** request is mitigated.
2. **(TMUI)** Confirm the difference in **Security > Event Logs > Bot Defense > Bot Requests**: before-access shows mitigation on the *first* request; after-access shows the first request passed and the *second* mitigated.
3. *Configuration (BIG-IP).* Restore the proactive setting when done:

```
tmsh modify security bot-defense profile lab-bot-defense class-overrides  
modify { Browser { verification { action browser-verify-before-access } } }
```

Task 4: Bot Exceptions by Category and Signature

The standalone profile allows specific bots while blocking others (Search Engine / Site Monitor none; Crawler alarm; HTTP Library / DOS Tool block).

1. **(kali)** Send a known search-engine user-agent and confirm it is **allowed** (Search Engine category = none):

```
curl -so /dev/null -w "%{http_code}\n" \  
  -A "Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com  
  /bot.html)" \  
  http://10.1.10.74/
```

2. **(kali)** Send an HTTP-library user-agent and confirm it is **blocked**:

```
curl -so /dev/null -w "%{http_code}\n" \
-A "python-requests/2.28.0" \
http://10.1.10.74/
```

3. (TMUI) Check classification in **Security > Event Logs > Bot Defense > Bot Requests** — note the *Bot Signature* and *Bot Category* columns.

Task 5: Search-Engine Verification — Masquerade, Verified, and Rate-Limiting Unknown

A client that merely *claims* to be Googlebot (via User-Agent) matches the Google search-engine signature and becomes a **candidate** Trusted Bot. Bot Defense then **verifies** it with forward-confirmed reverse DNS (FCrDNS):

1. Reverse-DNS (PTR) lookup of the **source IP** — the hostname must be in the search engine's domain (googlebot.com).
2. Forward-confirm (A) lookup of that hostname — it must resolve back to the same source IP.

The outcome drives classification:

- **Pass** → **Trusted Bot** (action *Alarm*; allowed). Trusted bots are allowed by design — they **cannot be blocked or rate-limited** (action *rate-limit* is rejected on Trusted Bot).
- **Fail** → **Malicious Bot**, anomaly *Search Engine Verification Failed* → **blocked regardless of class configuration**. Copying a good bot's UA does not make you trusted — it gets you blocked.

Rate-limiting, meanwhile, is a mitigation for the **Unknown** class (it is the Balanced-template default for Unknown), *not* for trusted/verified bots. This task shows all three: a masquerader blocked, a verified bot allowed, and the Unknown class rate-limited.

Step 0 — Stand up DNS for verification

Because verification is reverse-DNS, BIG-IP needs a **DNS Resolver** reachable over the TMM /data-plane interface, and a DNS server that answers the googlebot PTR/A records. In this lab, **kali** hosts a tiny responder (scripts/setup/labdns.py) that answers exactly:

- PTR 10.1.10.100 → crawl-lab-100.googlebot.com and the matching forward A back to 10.1.10.100 (so 10.1.10.100 **verifies**)
- nothing for 10.1.10.200 (so 10.1.10.200 stays a **masquerader**)

1. (**kali**) Start the responder (runs on Python 2 or 3; needs root for port 53):

```
cp /home/ec2-user/lab/scripts/setup/labdns.py /home/ec2-user/labdns.py
pkill -f labdns.py 2>/dev/null
nohup python /home/ec2-user/labdns.py >/var/log/labdns.log 2>&1 &
ss -ltnp | grep ':53' # confirm it is listening on 0.0.0.0:53
tail -f /var/log/labdns.log # watch queries arrive (leave running)
```

If :53 is already held, stop the local stub first: `systemctl stop systemd-resolved`.

2. (**BIG-IP**) Point a DNS Resolver's catch-all zone at kali. This blueprint ships a resolver named `f5-aws-dns` (used for Device ID+/idservice) — add a `.` forward zone to it so reverse lookups go to kali, and disable query-case randomization (dnsmasq-style servers lowercase replies, which breaks verification). (TMUI) **Network > DNS Resolvers > DNS Resolver List > f5-aws-dns:** add Forward Zone `.` with nameserver `10.1.10.100:53`, and set **Randomize Query Character Case** = *disabled*. (CLI):

```
tmsh modify net dns-resolver f5-aws-dns randomize-query-name-case no
forward-zones add { . { nameservers add { 10.1.10.100:53 { } } } }
tmsh save sys config
```

Note Resolver selection. Bot Defense uses the **first** DNS Resolver on the route domain associated with

the traffic. If several resolvers exist on route-domain 0, consolidate so the chosen one carries the . → kali zone; keep specific zones (e.g. amazonaws.com, idservice.net) pointing at their real servers. Lookups egress the **TMM/data-plane** interface (sourced from the BIG-IP self-IP, e.g. 10.1.10.9), *not* management — so the resolver target must be reachable over TMM (kali's 10.1.10.100 is on the connected client subnet).

Step 1 — Masquerade: a fake Googlebot is blocked

1. (kali) Send the Googlebot UA from 10.1.10.200 (which has **no** matching PTR):

```
curl --interface 10.1.10.200 -A "Mozilla/5.0 (compatible; Googlebot/2.1;
+http://www.google.com/bot.html)" -so /dev/null http://10.1.10.74/
```

2. (TMUI) In **Security > Event Logs > Bot Defense > Bot Requests**, the entry shows **Bot Class: Malicious Bot, Detected Anomalies: Search Engine Verification Failed, Mitigation Action: Block, Request Status: Denied**.

Masquerading Googlebot blocked

Figure 3.6. Fake Googlebot from an unverified IP → Malicious Bot, verification failed, denied.

Step 2 — Verified: a real-looking Googlebot is trusted

1. (kali) Send the same UA from 10.1.10.100 (PTR faked to crawl-lab-100.googlebot.com):

```
curl --interface 10.1.10.100 -A "Mozilla/5.0 (compatible; Googlebot/2.1;
+http://www.google.com/bot.html)" -so /dev/null http://10.1.10.74/
```

Watch the responder log — both lookups must answer (PTR then forward-confirm):

```
q 100.10.1.10.in-addr.arpa type=12 from 10.1.10.9 -> ANSWER
q crawl-lab-100.googlebot.com type=1 from 10.1.10.9 -> ANSWER
```

2. (TMUI) The entry now shows **Bot Class: Trusted Bot, Bot Categories: Search Engine, Detected Anomalies: N/A, Mitigation Action: Alarm** — allowed. A verified search engine is trusted; it is neither blocked nor rate-limited.

Verified Googlebot classified Trusted Bot

Figure 3.7. Same UA from a PTR-verified IP → Trusted Bot, Search Engine, allowed (Alarm).

Note Negative cache. The resolver caches results, including failures. The first request right after you fix DNS may still show the old *verification failed* verdict until the cached entry ages out — re-run, or test from a source that has not been classified yet.

Step 3 — Rate-limit the Unknown class

Rate limiting is available on the **Unknown** class (not on trusted/verified bots). In the **Strict** template Unknown defaults to **Block**; switch it to **Rate Limit** with a low cap so a single lab client trips it.

1. (TMUI) **Bot Mitigation Settings → Unknown → Rate Limit for 5 transactions per second. (CLI)** — note add (creates the override) and that action `rate-limit` is valid only for Unknown:

```
tmsh modify security bot-defense profile lab-bot-defense class-overrides add {
Unknown { mitigation { action rate-limit rate-limit-tps 5 } } }
```

Unknown class set to Rate Limit for 5 tps

Figure 3.8. Bot Mitigation Settings — Unknown = Rate Limit for 5 transactions per second.

2. (**kali**) Drive traffic that classifies as **Unknown** — use a neutral UA (a tool UA like `curl/8.x` matches an HTTP-Library signature and lands in *Untrusted*). Confirm the class on one request first, then exceed 5 TPS:

```
curl --interface 10.1.10.200 -A "labclient/1.0" -so /dev/null http://10.1.10.74/
for i in $(seq 1 300); do curl --interface 10.1.10.200 -A "labclient/1.0" -so /dev/null http://10.1.10.74/ & done; wait
```

3. (**TMUI**) Above 5 TPS, over-limit requests are denied. Open an entry's **Mitigation Action** detail: **Configured Action: Rate Limit, Actual Action: TCP Reset, Actual Action Reason: Rate Limit Request (TCP Reset), Request Status: Denied**. Traffic *up to* the cap is allowed — rate limiting throttles, it does not flat-block.

Unknown over-limit request rate-limited via TCP reset

Figure 3.9. Over-limit Unknown request → Rate Limit enforced as TCP Reset (Denied).

Step 4 — Teardown

Restore the profile and remove the lab DNS plumbing so other labs are unaffected:

```
tmsh modify security bot-defense profile lab-bot-defense class-overrides modify {
Unknown { mitigation { action block } } }
tmsh modify net dns-resolver f5-aws-dns forward-zones delete { . }
tmsh save sys config
```

On kali, stop the responder: `kill -f labdns.py`.

Task 6: Custom Bot Signature and Policy Exception

Beyond the built-in signatures, you can author a **custom bot signature** to catch a specific tool, then add a **policy exception** so a trusted case that would otherwise match is let through.

Step 1 — Create the custom signature (Configuration — BIG-IP)

1. **Security > Bot Defense > Bot Signatures > Bot Signatures List > Create** (older builds: **Security > Options > DoS Protection > Bot Signatures List**).
2. Configure:

Field	Value
Name	lab-scraper
Category	DOS Tool (a category the profile blocks)
Rule (Simple Edit Mode)	User-agent → contains → labscraper
Risk	Medium

CLI equivalent:

```
tmsh create security bot-defense signature lab-scraper category "DOS Tool"
risk medium user-agent { match-type contains search-string labscraper }
```

Important New custom signatures are placed in **staging** (logged, not blocked) for the profile's *Enforcement Readiness Period*. To see it **block** immediately, enforce the signature (or set the readiness period to 0) on the profile; otherwise the first validation shows the match in the logs while still passing traffic.

Step 2 — Validate detection

1. **(kali)** Send the matching user-agent:

```
curl -so /dev/null -w "%{http_code}\n" -A "labscraper/1.0" http://10.1.10.74/
```

Expected (once enforced): **blocked**. In staging: allowed but logged.

2. **(win-client)** Browse `http://10.1.10.74/` normally in the Windows client's browser. Expected: **allowed** — a real browser's user-agent does not contain `labscraper`, so the custom signature does not match. (Confirms the signature is specific, not blanket.)
3. **(TMUI)** In **Security > Event Logs > Bot Defense > Bot Requests**, confirm the kali request matched **Bot Signature = lab-scraper** and the win-client request did not.

Step 3 — Add a policy exception (Configuration — BIG-IP)

Suppose an internal tool legitimately uses that user-agent. Add a **Signature Exception** so the policy no longer mitigates `lab-scraper`:

- **UI:** open the `lab-bot-defense` profile > **Signature Exceptions** (or *Bot Mitigation Exceptions*) > **Add**, select `lab-scraper`, action **None**, save.
- **CLI:**

```
tmsh modify security bot-defense profile lab-bot-defense signature-overrides  
add { lab-scraper { action none } }
```

Step 4 — Validate the exception

1. **(kali)** Repeat the matching request:

```
curl -so /dev/null -w "%{http_code}\n" -A "labscraper/1.0" http://10.1.10.74/
```

Expected: **allowed** now — the exception overrides the category block for this signature.

2. **(TMUI)** In **Bot Requests**, the same signature now shows action **none / allowed** instead of **blocked**.
3. *Cleanup (BIG-IP), optional* — remove the exception to restore blocking:

```
tmsh modify security bot-defense profile lab-bot-defense signature-overrides  
delete { lab-scraper }
```

Task 7: The JS Challenge — Automation Is Gated, a Real Browser Passes

Tasks 2–3 showed *when* the challenge is issued. This task shows the *outcome*: non-browser automation never solves it and is classified/mitigated, while a real browser solves it transparently and is classified **Browser**.

Note Why the demo uses two sources. Solving a modern Bot Defense JS challenge needs a current browser engine. The lab's kali image cannot run one (Firefox ESR 45 has no headless mode; a 2025 Chrome needs a newer glibc/NSS than the image provides), so the **automation** half runs from kali and the **real-browser** half runs from **superjump**, whose in-browser Firefox is current. A *scripted headless* browser that solves the challenge yet is still flagged as automation requires a modern engine — `scripts/attack/js-challenge-client.sh` carries a commented, version-guarded Chrome-for-Testing block that activates if the image is ever upgraded.

Part A — Automation from kali (gated)

1. **(kali)** Run the client against `vs-lab-bot`:

```
bash /home/ec2-user/lab/scripts/attack/js-challenge-client.sh http:  
//10.1.10.74/
```

It shows `curl` receiving the **JS challenge page** (not the app), then sends three client identities and prints the classification to expect for each.

2. (TMUI) In **Security > Event Logs > Bot Defense > Bot Requests**, confirm each kali source:

- bare Mozilla/5.0 → **Malicious Bot / Exploit Tool** → blocked (a sparse UA matches an exploit-tool signature)
- labclient/1.0 → **Unknown** → rate-limited, if Unknown = Rate Limit (Task 5)
- Googlebot UA → **Trusted Bot** if the source PTR verifies (Task 5, Step 2), or **Malicious Bot / Search Engine Verification Failed** if not (Step 1)

None of these solve the challenge — a script has no JS engine.

Automation from kali is classified and mitigated

Figure 3.10. Non-browser automation from kali → classified and mitigated; it never reaches the app.

Part B — A real browser from superjump (passes)

1. (**superjump**) Open the in-browser Firefox (**ACCESS > FIREFOX**) and browse to `http://10.1.10.74/`. It renders the challenge briefly, runs the JS, receives a TS* cookie, and loads the Hackazon app. Reloading does not re-challenge.
2. (TMUI) In **Bot Requests**, find the entry from superjump's source (10.1.1.8): **Bot Class: Browser, Request Status: Accepted, Detected Anomalies: N/A**. The browser also pulls page assets (`/favicon.ico`, `/fonts/...`, `/products_pictures/...`) — all **Accepted / Browser** — which a script never does.

Real browser from superjump classified Browser and accepted

Figure 3.11. superjump's real browser solves the challenge → Browser (verified); app and assets accepted.

The lesson. Identical requests, opposite outcomes: the JS challenge is transparent to a real browser and an impassable gate to a script. Passing the challenge is necessary but not sufficient — Proactive Bot Defense also applies browser-integrity and signature checks, so a JS-capable automation (a scripted headless browser) can still be flagged as **Suspicious Browser**. Demonstrating that live needs a modern headless engine this lab image cannot run (see the note above).

Replay test — cross-source cookie binding

A solved TS* cookie is signed and time-stamped (it cannot be forged or altered), but whether it can be **replayed** from another client depends on how strictly the deployment binds it (source IP, User-Agent, TLS fingerprint) and its TTL. Capture a real cookie from superjump, then replay it from kali.

1. (**superjump**) After Part B loads the app, open **DevTools > Storage/Application > Cookies** for `http://10.1.10.74` and copy the TS* cookie name and value.
2. (**kali**) Replay that superjump-earned cookie from two kali sources (kali is a *different host* than the solver):

```
TS='TSPD_101=<paste-the-value>'
curl -s -o /dev/null -w 'kali .100: %{http_code}\n' -A 'Mozilla/5.0' --cookie "$TS" --interface 10.1.10.100 http://10.1.10.74/
curl -s -o /dev/null -w 'kali .200: %{http_code}\n' -A 'Mozilla/5.0' --cookie "$TS" --interface 10.1.10.200 http://10.1.10.74/
```

Interpret (cross-check in **Bot Requests**):

- **both challenged/blocked** → the cookie is bound to the solver's source and/or fingerprint — it is **not replayable** to another host.
- **either passes** → the cookie is **replayable** to a different host within its TTL (the classic scraper reuse).

(The automated `bot-cookie-replay.sh` solves and replays in one step, but it needs a modern

headless browser — so on this image use the manual capture above.)

This is why F5 layers Device ID+, browser-integrity checks, and Distributed Cloud Bot Defense (whose telemetry is IP/browser-bound with **no** reusable token) on top of the challenge cookie.

Questions

- A mobile app using a WebView cannot reliably solve the browser JS challenge. Which verification action (or class/allowlist entry) keeps PBD active for browsers while letting the app through?
- Your custom `lab-scraper` signature matches on user-agent. Give two ways an attacker evades it, and which other Module 3 control would still catch each.
- The Signature Exception in Step 3 disables `lab-scraper` globally. How would you instead except *only* a trusted URL or source IP while still blocking that user-agent everywhere else?
- A real browser (superjump) solved the JS challenge and earned a cookie, while kali automation could not. Why is passing the challenge *not* sufficient on its own, and what additional Bot Defense signals would still distinguish a scripted headless browser from a real one?
- In the replay test, a cookie earned by the real browser (superjump) is replayed from kali. What does each outcome — challenged vs. passed — tell you about how the cookie is bound, and why does F5 prefer IP/fingerprint binding over a freely replayable token?

3.5 Instructor Answer Key

Model answers to the **Questions** at the end of each lab. Instructor material — keep it out of the student hand-out. Answers are talking points, not the only correct phrasing; encourage discussion.

3.5.1 Module 1 — iRules

Lab 1 — Per-IP rate limiting

Shared-NAT users (corporate proxy, 100+ employees): they all share one source IP, so the per-IP counter sums the whole office. Their combined rate trips the threshold and legitimate users get blocked — the classic weakness of per-IP limiting: it can't tell individuals apart behind one address. Fixes: key on an L7 identity (XFF, session cookie, JWT claim) instead of IP, raise/allow-list known proxy ranges, or move to behavioral/bad-actor detection (Module 3).

Warning header at 80%: compare the counter to $0.8 \times \text{threshold}$. Between 80% and 100%, set a flag and add a response header (e.g. `X-RateLimit-Warning`) in `HTTP_RESPONSE` while still forwarding; only `HTTP::respond 429` at 100%.

Why `table incr`: it is atomic. A `lookup then set` is a read-modify-write race — two concurrent requests can both read `N` and both write `N+1`, undercounting. `table incr` increments in one operation, so concurrent requests are counted correctly (and it's one table op instead of two).

Lab 2 — Per-URI rate limiting

Query parameters: depends on the key. Keying on `[HTTP::uri]` (includes the query string) makes `/search?q=a` and `/search?q=b` *different* buckets — so an attacker just varies the query to evade the limit. Key on `[HTTP::path]` (path only) so all queries to `/search` share one counter.

Different thresholds per path: map `path` → `threshold` (a switch on `HTTP::path` setting a `$limit` variable, or a data-group lookup as in Module 2 Lab 3), then run the generic counter logic against `$limit`.

Lab 3 — Concurrent connection limit

vs. the built-in VS Connection Limit: the built-in limit is a single global L4 cap on concurrent connections to the virtual server — blunt, all clients combined. The iRule tracks concurrent connections *per source IP*, so one abusive client is capped without touching everyone else, with L7 awareness.

Built-in is faster/simpler; the iRule is granular.

What it misses: a high request-rate attack that opens and closes connections quickly (few concurrent at any instant), or many requests over one reused keep-alive connection. Request-rate limiting handles that — Lab 1 (per-IP) and Lab 4 (sliding window).

Lab 4 — Sliding window

Memory upper bound: the timestamp-list grows with clients × requests-in-window, so TMM table memory scales badly at internet scale. Apply it to bounded populations (hundreds to low thousands) or to specific high-value URIs; for large scale use cheap fixed-window counters (one integer per client) or the DoS profile's behavioral detection.

Combine with per-URI: make the table key a composite of client IP + path (sw_<ip>_<path>) so each (client, endpoint) pair gets its own window, then apply per-endpoint thresholds.

Lab 5 — Custom L7 DoS signature (header-absence)

Global vs per-IP cap: change the counter key to a single fixed key so all header-less traffic shares one ceiling. Per-IP throttles each offender independently (one bad client can't starve others); global is better when *any* header-less traffic is inherently suspect, or when attackers spread across many IPs so per-IP never trips.

Split detection (LTM policy) from mitigation (iRule): an LTM policy with an `http-header ... missing condition matches header-less requests` and hands them to `policy-triggered-rate-limit.tcl` (via a marker header / rate tier). Gains: detection is declarative and auditable in the policy, evaluation is efficient, and one shared mitigation iRule serves many detection rules — separation of “what to match” from “how to throttle.”

Requiring a token value: read `[HTTP::header X-Client-Token]` and validate the value (known secret, HMAC, signed/rotating token) rather than just presence. That crosses from a cheap DoS shape-check into authentication — which needs secret management, rotation, and replay protection, should fail *closed*, and belongs with the app/Access, not a fail-open DoS iRule.

3.5.2 Module 2 — LTM Policies

Lab 1 — Rate filter / bandwidth

When bandwidth shaping beats request-rate limiting: when bytes matter more than request count — large downloads, media/streaming, protecting a backend/egress link, or when response sizes vary so a few big responses saturate the link at low request rate. Bandwidth shaping caps bytes/sec; request-rate caps requests/sec.

Burst size: a rate class has a sustained rate plus a burst (bucket depth). Set the burst to a few seconds' worth of traffic so legitimate bursts (a page pulling many assets) pass, while sustained traffic is held to the configured rate — token-bucket style: rate = refill, burst = depth.

Lab 2 — Policy + iRule

Why strip ``X-RateLimit-Profile`` before the pool: it's an internal control header (policy sets it, iRule reads it to pick a tier). Leaking it exposes policy internals, and if a client could set it they could spoof `X-RateLimit-Profile: permissive` to pick a lax tier. Strip it egress (and normalize any client-supplied copy on ingress) to protect the control channel.

No matching rule / missing header: with no matching rule and no default, the request passes unmodified (no tier assigned). The iRule must handle a missing/empty header deliberately — apply a defined default tier (or pass). Decide the default on purpose: fail-open favors availability; a default tier is safer. Always handle the empty case so untagged traffic isn't accidentally blocked *or* accidentally unlimited.

Lab 3 — Data-group driven

Overlapping prefixes: F5 string data-groups are evaluated as a prefix tree, so `class match -value $uri starts_with rate_limit_paths` returns the *longest* matching prefix — `/user/login` matches the `/user/login` entry rather than a broad `/user` when both exist. If precedence is critical,

keep entries non-overlapping or handle specific-before-general explicitly, and test — don't assume ordering.

Data-group vs external source: a data-group is in-memory, fast, and dependency -free but relatively static (updated via config change/sync). An external source (runtime iControl REST, sideband) is dynamic and centrally managed but adds latency, a failure mode, and complexity. Use data-groups for stable lists; use an external feed for fast-changing/centrally-owned lists — or reconcile the two by refreshing the data-group from the external source on a schedule.

Lab 4 — Reject bad paths

TCP reset/drop vs HTTP 403: a silent drop gives the scanner nothing — no page, no banner, no confirmation the path exists or that a control is present, and it burns their time on timeouts. A 403 confirms active filtering (useful recon). For bait/honeypot paths you want zero information leakage.

Feed Module 3 Bot Defense: on a bad-path hit, write the source IP to a shared address-list / IP-Intel-
ligence category / shun list (via `table`, a sideband, or AFM), and have Bot Defense / IP Intelligence block sources in that list — the honeypot-promotes-to-blocklist pattern.

3.5.3 Module 3 — ASM / Advanced WAF

Lab 1 — TPS-based

What the baseline is: the auto-learned *normal* transactions-per-second for the URL/source, measured as a moving average over a rolling learning period. “Increased by 500%” means mitigate when the current rate is 5× that learned baseline (subject to any absolute floors/ceilings you set) — a relative threshold, not a fixed number.

Per-URL thresholds: in the DoS profile's TPS-based detection, define URL-specific entries (e.g. `/user/login`) with stricter TPS/percentage thresholds than the site-wide default.

Lab 2 — Stress-based

Legit automated report (200 req/10s): TPS-based would likely block it — it exceeds a rate threshold regardless of impact. Stress-based only mitigates when the *server is actually stressed* (latency climbing), and then targets the heaviest contributors to that latency. If the server absorbs the report without latency rising, stress-based leaves it alone — it's more forgiving of harmless bursts.

De-escalation too short: if mitigation is released as soon as latency normalizes — but latency only dropped *because* mitigation was working and the attack is ongoing — the attack resumes, latency spikes, mitigation re-escalates, and you get flapping (on/off) with intermittent degradation. A longer de-escalation confirms the server is genuinely healthy and the attack has actually stopped before releasing.

Lab 3 — Behavioral DoS (BADoS)

Health recovers mid-attack: BADoS detected the anomaly, generated dynamic signatures for the attack traffic, and started mitigating (first global rate-limiting to protect the server, then targeted signature/bad-actor drops) — so the server recovers even though the attacker keeps sending, because the bad traffic is now being dropped. Unlike a static limit (a fixed threshold that blunt-blocks everyone over it), BADoS learns normal, isolates the anomalous pattern, and mitigates just that — no preset threshold.

Dynamic signatures vs bad-actor greylist: signatures describe the *attack traffic's shape* and mitigate any request matching the pattern regardless of source (catches new IPs); the greylist identifies *misbehaving source IPs* and throttles those sources. You see signatures when the attack has a distinct request pattern (Request Signatures Detection on); you see greylisting when sources are identifiable (Bad Actor Detection on). Often both — signatures restore health fast, bad-actor then pins the sources.

Persisting a dynamic signature: gain — it survives the attack, so the same attack is mitigated instantly next time without re-learning, and it's shareable and auditable. Risk — a signature born in a noisy, mixed attack may be over-broad and match legitimate traffic, causing false positives when made always-on. Review its predicates before persisting; persist only vetted signatures.

Lab 4 — Custom persistent DoS signatures

Learn Only / Detect Only vs Mitigate: use them to validate a new signature without risking false positives — watch what it *would* catch on real traffic before enforcing. “Use Approved Signatures Only” makes rollout safer: only manually reviewed/approved signatures actually mitigate, so an auto-generated or unvetted signature stays inert until you approve it — preventing an over-broad signature from blocking legit users. Stage → review → approve → Mitigate.

Keeping only `http.user_agent_header_exists eq false`: it matches *every* request with no User-Agent — far too broad. Many legitimate non-browser clients omit UA: uptime/health monitors, load-balancer probes, internal scripts, some API integrations and IoT. You’d generate false positives. (It’s also trivially evaded — an attacker just adds any UA.) That’s why the lab ANDs it with `http.request.method` and `http.uri_len` to pin the actual attack shape.

Why `origin` provenance matters: when auditing which signatures are enforcing, `origin` says how each came to be — `dynamic-bdos` = machine-generated by BADoS from an observed attack (automatic, possibly broad); `user-defined` = deliberately authored by a person (intentional, with a traceable rationale/owner). Provenance drives trust and change control: auto-generated signatures warrant more scrutiny for over-breadth, and user-defined ones have an owner you can ask when one causes a false positive. Lab 5 — Proactive Bot Defense ~~~~~

Mobile WebView can’t solve the JS challenge: keep the browser JS challenge on the Browser class, but handle the app differently — use mobile integrity verification / the F5 Anti-Bot Mobile SDK for the Mobile class, a challenge-free/header-based verification for that path, or allowlist the app’s API endpoints or user-agent so PBD skips the JS challenge there while still challenging browsers.

Evading the `lab-scraper` UA signature: (1) spoof a real browser user-agent — the UA-contains signature no longer matches, but behavioral/bad-actor detection or the rate limits still catch the volumetric behavior; (2) run a headless browser that solves the JS challenge (or go low-and-slow) — the proactive JS challenge / verify-before-access catches a non-solving client and rate limits catch the rate. The custom signature is one narrow layer; the challenge + behavioral + rate-limit layers cover its gaps.

Scope the exception to a URL/source: instead of a global Signature Exception (action none everywhere), add a `whitelist` entry scoped to the trusted URL or source IP (`disable-mitigation`) so only that traffic bypasses the signature, while the signature still blocks the user-agent everywhere else. Whitelist = scoped bypass; signature exception = global disable.

Headless browser passed the challenge — why it isn’t “legitimate”: solving the JS challenge only proves the client runs JavaScript and handles cookies — which headless Firefox/Chrome, Selenium, and Puppeteer all do. It filters out plain scripts (curl), nothing more. Proactive Bot Defense pairs the challenge with browser-integrity/anomaly checks and bot signatures — headless/automation fingerprints, missing or abnormal browser attributes, behavioral signals, and device-ID consistency — so a JS-capable bot can still be flagged as a Suspicious Browser or matched to an automation signature. With Block Suspicious Browsers on, that headless client can be caught despite earning a cookie.

Cookie-replay outcomes and why F5 binds the cookie: `TSPD_101` is signed and time-stamped, so it can’t be forged or modified and it expires — but replay *across hosts* is governed by what it’s bound to.

- *same PASSED, alt CHALLENGED* → the cookie is bound to the source IP; a copy presented from `10.1.10.200` fails and that host is re-challenged.
- *both PASSED* → it isn’t strictly IP-bound, so it is replayable to any host within its TTL — the classic “solve once, reuse on cheap HTTP clients” scrape.
- *both CHALLENGED* → it’s also bound to attributes `curl` can’t reproduce (User-Agent / TLS fingerprint), so even same-host replay is rejected.

F5 prefers IP/fingerprint binding because a freely replayable token lets an attacker solve the challenge once (with a real/headless browser) and distribute the cookie to a whole botnet of cheap scripted clients — defeating the point of the challenge. Binding, plus Device ID+ and (in Distributed Cloud Bot Defense) non-reusable telemetry, keeps the “proof of solving” tied to the client that actually solved it.

